

INFORME DE AVANCE N°2:

DESARROLLO DEL PROYECTO

Contrabajo:

“Tu instrumento para trabajar”

Asignatura: Taller Aplicado de Programación (TPY1101).

Proyecto: Contrabajo.

Integrantes: Cristóbal Barrientos, Rubén Parada y Tomás Zapata.

Profesor/a: Marcela Gonzalez.

INFORME DE AVANCE N°2:	1
DESARROLLO DEL PROYECTO	1
1. INTRODUCCIÓN Y EQUIPO	4
1.1 Resumen Ejecutivo	4
1.2 Identificación del Equipo	5
1.3 Visión del Proyecto	6
1.4 Objetivos SMART	6
1.5 Planificación del Proyecto	7
2. CONTEXTO Y PROBLEMÁTICA	8
2.1 Contexto del problema	8
2.1.1 Mercado Objetivo	8
2.1.2 Perfil Demográfico del Usuario	9
2.2 Descripción de la Problemática	11
2.3 Análisis Causa-Efecto	12
2.4 Estado del Arte y Homologación	13
2.5 Situación Inicial (As-Is)	14
2.6 Justificación del proyecto	15
2.6.1 Historias de usuario	16
3. OBJETIVOS Y ALCANCE (IL 1.2)	18
3.1 Objetivo General	18
3.2 Objetivos Específicos	19
3.2.1 Conexión Cliente–Trabajador	19
3.2.2 Lógica de Negocio (Backend)	19
3.2.3 Seguridad y Confiabilidad	19
3.2.4 Arquitectura de Datos	19
3.2.5 Experiencia de Usuario (UX/UI)	19
3.2.6 Infraestructura y Despliegue	19
3.2.7 Gestión Ágil del Proyecto	19
3.3 Alcance Técnico	20
3.3.1 Entregables del Proyecto	20
3.3.2 Supuestos del Proyecto	20
3.3.3 Restricciones del Proyecto	20
4. PROPUESTA DE SOLUCIÓN Y DISEÑO CONCEPTUAL (IL 1.1 / IL 1.4)	21
4.1 Definición de Requerimientos	21
4.1.1 Identificación de Actores	22
4.1.2 Casos de uso	23
4.1.3 Diagramas de Casos de Uso	25
4.1.4 Modelo Entidad-Relación (MER)	27
4.1.6 Diagramas UML	32
4.1.6.1 Introducción a los Diagramas UML	32
4.1.6.2 Diagrama de Paquetes	33
4.1.6.3 Diagrama de Secuencia	34
4.1.6.4 Diagrama de Despliegue	35
4.2 Requisitos Funcionales (RF) - Reglas de Negocio	36
4.3 Requisitos No Funcionales (RNF) - Atributos de Calidad	37
4.4 Diseño de Interfaz (Mockups)	38
4.4.1 Contrabajo Prototipo v1	38
4.5 Arquitectura y Patrones	39
4.5.1 FrontEnd: Arquitectura MVVM (Model-View-ViewModel)	39
4.5.2 BackEnd: Microservicios y Patrón de Capas	39
4.6 Estrategia Cloud	40
4.6.1 Proveedor Seleccionado: Microsoft Azure	40
4.6.2 Modelo de Servicio: IaaS (Infrastructure as a Service) usando Docker	40
4.6.3 Factibilidad y Plazos	41
5. DESCRIPCIÓN TÉCNICA DEL ENTORNO Y AMBIENTE DE PRUEBAS	42
5.1 Infraestructura Tecnológica	42
5.1.1 Frontend (Desarrollo Móvil Nativo)	42
5.1.2 Backend (Lógica de Negocio)	43
5.1.3 Base de Datos (Persistencia de Datos)	44
5.1.4 Infraestructura y Despliegue (Cloud)	45
5.1.5 Configuración y Comunicación entre Microservicios	46
5.1.6 Seguridad y Gestión de Autenticación	48

5.1.7 Estrategia de Escalabilidad y Alta Disponibilidad	49
5.2 Repositorio y Versionamiento	51
5.2.1 Estrategia de Versionamiento Semántico	51
5.2.2 Organización de Repositorios y Microservicios	53
5.3 Ambiente de Desarrollo	55
5.3.1 Entorno Local de Desarrollo	57
5.3.2 Estrategia de Respaldo y Recuperación (Backup & Restore)	60
5.3.4 Configuración del Servidor	62
5.4 Desarrollo Funcional Implementado	63
6. METODOLOGÍA Y PLANIFICACIÓN (IL 1.4)	67
6.1 Marco de Trabajo Ágil	67
6.2 Mapeo de Iteraciones y Cronograma de Trabajo	67
7. MECANISMOS DE CONTROL Y CALIDAD	69
7.1 Certificación de Avances	69
7.2 Plan de Pruebas Inicial	70
7.3 Plan de Pruebas Detallado	71
7.3.1 Pruebas Operativas, de Validación y Verificación	71
7.3.2 Registro de Evidencias Empíricas de Ejecución	72
7.3.3 Evidencia de Pruebas Unitarias Automatizadas y Cobertura de Código	73
8. CONCLUSIÓN	75
9. BIBLIOGRAFÍA	75
10. ANEXOS	76
Entrevistas y Mapas de empatía (levantamiento de información)	76
Mapas de empatía:	76
Historias de usuario:	78

1. INTRODUCCIÓN Y EQUIPO

1.1 Resumen Ejecutivo

En el contexto actual, el mercado laboral atraviesa una transformación acelerada impulsada por el avance de tecnologías digitales, especialmente la inteligencia artificial. Si bien estas herramientas han permitido automatizar diversas tareas, su impacto se ha concentrado principalmente en actividades digitales, mientras que los oficios técnicos presenciales (como gasfitería, electricidad o mecánica) continúan dependiendo en gran medida de la intervención humana debido a la complejidad de los entornos físicos y la variabilidad de los problemas. Esto mantiene vigente la demanda por este tipo de servicios, aunque en un escenario que exige mayor adaptación, formalización y visibilidad.

Paralelamente, en países como Chile y gran parte de Latinoamérica, se observa una alta presencia de trabajadores independientes en el rubro de servicios técnicos. Muchos de ellos operan en condiciones de informalidad (Ministerio de Economía & Instituto Nacional de Estadísticas, 2023), lo que da lugar a una oferta fragmentada y con escasa presencia digital. Esta situación dificulta el acceso a oportunidades laborales para los prestadores, mientras que los clientes enfrentan incertidumbre respecto a la calidad del servicio y la falta de mecanismos confiables de verificación. A pesar del aumento en el acceso a internet, persiste una brecha significativa en el uso efectivo de herramientas digitales por parte de estos trabajadores.

En respuesta a esta problemática, se propone **Contrabajo**, una aplicación móvil orientada a la intermediación de servicios técnicos mediante un modelo tipo *marketplace*. La plataforma permite a los prestadores publicar y gestionar sus servicios, mientras que los clientes pueden explorarlos, comunicarse directamente y coordinar citas de manera sencilla. De este modo, se busca mejorar la visibilidad de los trabajadores, reducir los tiempos de búsqueda y aumentar la confianza en las transacciones.

La solución se implementa sobre una arquitectura moderna basada en microservicios desarrollados en Java, un frontend nativo en Kotlin y una infraestructura escalable desplegada en la nube de Microsoft Azure. Este enfoque garantiza un sistema robusto, flexible y preparado para el crecimiento. El presente informe describe la conceptualización del proyecto, su propuesta de valor, los requerimientos definidos y la arquitectura técnica que sustenta la solución.

1.2 Identificación del Equipo

El éxito de un sistema complejo no reside únicamente en la calidad de su código, sino en la sinergia y especialización del capital humano que lo construye. Bajo un enfoque de roles definidos y responsabilidades claras, presentamos la estructura organizacional encargada de transformar los requerimientos de negocio en una solución tecnológica de alto rendimiento, asegurando que cada etapa del ciclo de vida del desarrollo cuente con un liderazgo técnico especializado.

A continuación, se detalla la composición del equipo de proyecto, asignando responsabilidades alineadas con las competencias técnicas y perfiles de cada integrante:

Integrante	Rol y Responsabilidades	Perfil Profesional
Rubén Parada (Project Lead & Scrum Master)	Responsable de la gestión en Jira, diseño de interfaz (UI/UX), desarrollo Frontend y aseguramiento de calidad (QA/Testing).	Estudiante de Analista Programador con enfoque Fullstack y arquitectura. Motivado por construir soluciones estructuradas con base sólida en buenas prácticas y diseño centrado en el usuario.
Cristóbal Barrientos (Backend & Integration Specialist)	Responsable de la lógica de servidor, integración de APIs, documentación técnica y ejecución dentro del equipo de desarrollo.	Estudiante de Analista Programador con interés en Ciencia de Datos. Destaca por su adaptabilidad, rapidez de aprendizaje y motivación frente a desafíos técnicos.
Tomás Zapata (Product Owner & Data Architect)	Responsable de la visión del producto, diseño de bases de datos, modelamiento de diagramas UML y estrategia de despliegue en la nube (Cloud).	Estudiante de Analista Programador apasionado por el análisis profundo de los entornos, comprometido con la creación de impacto y la comprensión de sistemas en mundos de cambio constante.

1.3 Visión del Proyecto

Contrabajo busca consolidarse como una plataforma digital confiable y eficiente para la contratación de servicios técnicos, posicionándose como un punto central de conexión entre trabajadores independientes y clientes en el ámbito de servicios domésticos.

La visión del proyecto es contribuir a la transformación de un mercado actualmente fragmentado e informal hacia un ecosistema digital estructurado, donde la confianza, la trazabilidad y la eficiencia sean elementos fundamentales en cada interacción. Para ello, se incorporan funcionalidades como verificación de identidad, sistemas de reputación y geolocalización, orientadas a profesionalizar la oferta de servicios y mejorar la experiencia de contratación.

1.4 Objetivos SMART

Objetivo General

Desarrollar una aplicación móvil tipo *marketplace* que permita conectar clientes con prestadores de servicios técnicos, mediante una plataforma basada en arquitectura de microservicios, con el propósito de mejorar la eficiencia, visibilidad y confianza en el proceso de contratación, dentro del periodo definido para el desarrollo del proyecto.

Objetivos Específicos:

- Diseñar e implementar la arquitectura del sistema basada en microservicios (usuarios, servicios y comunicaciones), utilizando Java y Microsoft Azure, antes de la semana 5 del proyecto.
- Desarrollar e integrar las funcionalidades de gestión de usuarios, incluyendo registro, autenticación y recuperación de contraseña, asegurando su correcto funcionamiento antes de la semana 6.
- Implementar las funcionalidades principales del negocio, tales como publicación de servicios, búsqueda de prestadores y comunicación entre usuarios, antes de la semana 8.
- Desplegar la solución en un entorno cloud utilizando Microsoft Azure, garantizando la disponibilidad y accesibilidad del sistema antes de la entrega final.
- Validar el sistema mediante la ejecución de al menos 10 casos de prueba funcionales, documentando sus resultados y evidencias en el informe final.
- Gestionar el desarrollo del proyecto mediante la metodología ágil Scrum, realizando seguimiento de tareas a través de Jira y controlando el avance por medio de sprints durante todo el ciclo de desarrollo.

Los objetivos definidos se encuentran alineados con la planificación del proyecto, presentada en el apartado 6.2, donde cada hito técnico corresponde a un objetivo específico y permite evaluar el progreso del desarrollo de manera incremental.

1.5 Planificación del Proyecto

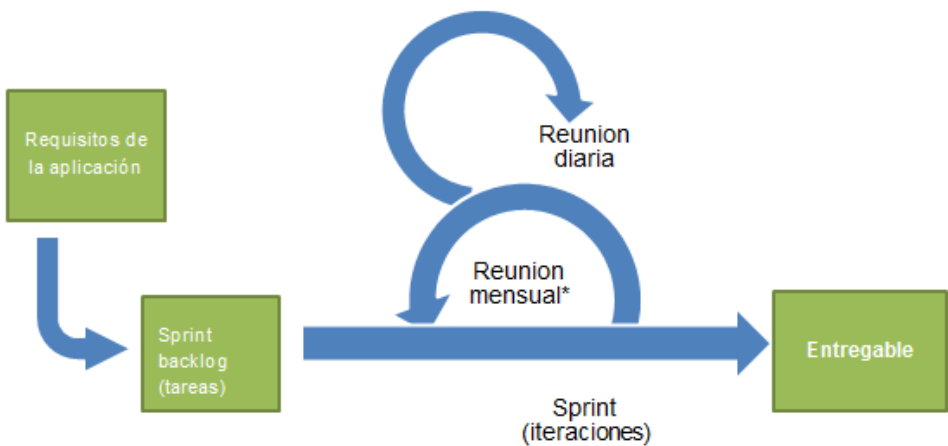
La planificación del proyecto Contrabajo se estructura en función de los objetivos definidos bajo el enfoque SMART, permitiendo una ejecución organizada, medible y alineada con los plazos académicos establecidos.

El desarrollo se divide en fases progresivas que abarcan desde la definición del problema hasta la implementación y validación del sistema funcional. Cada fase contempla entregables concretos que permiten evaluar el avance del proyecto de forma incremental.

En términos generales, la planificación considera:

- **Fase de Análisis y Diseño (Semanas 1–4):**
Definición de la problemática, levantamiento de requerimientos, elaboración de diagramas (UML y MER) y diseño de la arquitectura del sistema.
- **Fase de Desarrollo (Semanas 5–12):**
Implementación de microservicios en Java, desarrollo del frontend en Kotlin y construcción de la base de datos en Azure SQL.
- **Fase de Integración y Pruebas (Semanas 13–16):**
Validación del sistema mediante pruebas funcionales, integración de componentes y ajustes finales.
- **Fase de Entrega (Semana final):**
Despliegue en entorno cloud, documentación final y presentación del producto funcional.

Esta planificación se complementa con la metodología ágil Scrum, permitiendo la adaptación continua del proyecto mediante iteraciones cortas (sprints), seguimiento de tareas en Jira y validación constante de los avances.



2. CONTEXTO Y PROBLEMÁTICA

2.1 Contexto del problema

El mercado de servicios técnicos en Chile y Latinoamérica presenta una combinación de factores estructurales que dificultan la conexión eficiente entre oferta y demanda. Existe una amplia disponibilidad de trabajadores independientes y microemprendedores en rubros de oficios presenciales, sin embargo, gran parte de esta oferta opera en condiciones de informalidad y con baja presencia digital.

En Chile, la informalidad laboral alcanza aproximadamente un 27% (*Instituto Nacional de Estadísticas, 2024*), concentrándose principalmente en trabajadores por cuenta propia, donde la tasa de informalidad supera el 69%. Asimismo, se estiman cerca de 2 millones de microemprendedores, de los cuales más de la mitad opera de manera informal.

A esto se suma una brecha en el uso productivo de herramientas digitales, donde las microempresas presentan una menor adopción en comparación con empresas de mayor tamaño, limitando su capacidad de visibilizar servicios y captar clientes.

Este contexto genera un mercado con alta disponibilidad de servicios, pero con dificultades estructurales en términos de acceso, confianza y eficiencia en la contratación.

2.1.1 Mercado Objetivo

El mercado objetivo de la solución propuesta se compone de dos segmentos principales: la oferta de servicios técnicos y la demanda de dichos servicios.

Por el lado de la oferta, se identifican trabajadores independientes y microemprendedores que desempeñan oficios técnicos presenciales, tales como gasfitería, electricidad, carpintería, cerrajería, mecánica y servicios generales. Este segmento se caracteriza por operar principalmente bajo esquemas de autoempleo y, en muchos casos, en condiciones de informalidad, lo que limita su visibilidad, acceso a clientes y oportunidades de crecimiento.

Por el lado de la demanda, el sistema está orientado a personas naturales y pequeños negocios que requieren soluciones técnicas de forma rápida, confiable y segura. Estos usuarios enfrentan dificultades para encontrar prestadores confiables, comparar alternativas disponibles y evaluar la calidad del servicio, debido a la falta de mecanismos estructurados de verificación y reputación.

En este contexto, la aplicación *Contrabajo* se posiciona como un intermediario digital que facilita la conexión entre ambos segmentos, centralizando la oferta de servicios, permitiendo la interacción directa entre usuarios y proporcionando herramientas que mejoran la confianza y transparencia del proceso, tales como sistemas de valoración, gestión de citas y comunicación integrada.

2.1.2 Perfil Demográfico del Usuario

Este análisis se encuentra respaldado por el levantamiento de información realizado, incluyendo entrevistas, mapas de empatía e historias de usuario, disponibles en el apartado de anexos del presente informe.

Con el fin de definir de manera precisa el público objetivo de la aplicación Contrabajo, se establece el siguiente perfil demográfico basado en las características de los usuarios que interactúan en la plataforma, considerando tanto la oferta como la demanda de servicios técnicos.

Segmento Cliente (Demanda)

Característica	Descripción
Descripción	Corresponde a personas naturales y pequeños negocios que requieren servicios técnicos para el hogar o actividades comerciales menores.
Edad	25 a 55 años
Ubicación	Zonas urbanas y periurbanas, principalmente en la Región Metropolitana y otras ciudades principales de Chile
Nivel socioeconómico	Medio y medio-bajo
Nivel educativo	Educación media completa o superior
Acceso tecnológico	Uso frecuente de smartphones y aplicaciones móviles
Comportamiento	Necesitan soluciones rápidas y confiables, buscan reducir el tiempo de búsqueda de servicios, valoran la seguridad, reputación y transparencia en precios

Segmento Trabajador (Oferta)

Característica	Descripción
Descripción	Corresponde a trabajadores independientes y microemprendedores que ofrecen servicios técnicos presenciales.
Edad	20 a 60 años
Ubicación	Zonas urbanas con alta densidad poblacional
Nivel socioeconómico	Medio y medio-bajo
Nivel educativo	Variable (desde educación media hasta formación técnica)
Condición laboral	Principalmente trabajadores por cuenta propia
Acceso tecnológico	Uso de smartphones, pero con menor nivel de digitalización productiva
Comportamiento	Buscan aumentar su visibilidad y captar nuevos clientes, necesitan herramientas simples para gestionar servicios y presentan baja formalización digital

Conclusión del Perfil

El perfil demográfico identificado permite comprender de manera integral las características, necesidades y comportamientos de los usuarios que conforman el ecosistema de Contrabajo. Tanto clientes como trabajadores comparten el uso de dispositivos móviles como principal medio de acceso, lo que evidencia la necesidad de desarrollar una plataforma accesible, intuitiva y adaptable a distintos niveles de experiencia digital.

2.2 Descripción de la Problemática

En base al contexto previamente descrito, se evidencia una problemática central en la forma en que actualmente se realiza la búsqueda, contacto y contratación de servicios técnicos. A pesar de la alta disponibilidad de trabajadores en el mercado, la falta de digitalización, validación y organización de la información genera un proceso ineficiente, fragmentado y poco confiable tanto para clientes como para prestadores de servicio, situación consistente con los niveles de informalidad presentes en el mercado laboral (Instituto Nacional de Estadísticas, 2024).

Esta situación se traduce en un escenario crítico caracterizado por los siguientes puntos:

- **Inseguridad y Falta de Validación:** Al no existir una plataforma centralizada que verifique la identidad y antecedentes de los prestadores de servicios, tanto clientes como trabajadores se exponen a riesgos de seguridad personal y fraudes. No existe un mecanismo técnico que garantice que la persona que llega al domicilio es quien dice ser.
- **Ausencia de Trazabilidad y Geolocalización:** El modelo actual carece de herramientas de tiempo real. Los clientes no pueden visualizar la proximidad de los técnicos, y los trabajadores no cuentan con un flujo de trabajo digital que registre el inicio, desarrollo y cierre de una atención, perdiendo el control sobre su propia productividad.
- **Asimetría de Información y Reputación:** No existe un repositorio histórico y transparente de valoraciones (reviews) que sea auditable. Esto impide que los técnicos destacados escalen su negocio y que los clientes puedan tomar decisiones basadas en datos de calidad de servicio, fiabilidad y cumplimiento de plazos.
- **Brecha de Formalización Digital:** Muchos trabajadores de oficio quedan excluidos de las oportunidades laborales modernas por no contar con una plataforma que gestione su portafolio, disponibilidad de forma profesional, dependiendo de canales de comunicación obsoletos y desorganizados (como chats grupales o llamadas perdidas).

La problemática descrita no responde a eventos aislados, sino a una estructura de intermediación débil, poco digitalizada y sin mecanismos estandarizados de control, validación ni registro.

Para comprender en profundidad el origen de estas deficiencias, se hace necesario analizar las causas que las generan, diferenciando entre los síntomas visibles del problema y sus factores estructurales. A continuación, se presenta un análisis causa-efecto mediante un Diagrama de Ishikawa.

2.3 Análisis Causa-Efecto

Evidencia: Diagrama de Ishikawa o Árbol de Problemas.

Este diagrama clasifica las causas de un problema en categorías específicas (las "6M", adaptadas aquí a tecnología). Es ideal para mostrar cómo la **tecnología obsoleta** y los **malos métodos** convergen en el problema actual.

- **La Cabeza:** Representa el efecto o problema principal (ej. *"Descontrol en la gestión de servicios"*).
- **Las Espinas Principales:** Son las categorías de causas.
 - **Tecnología:** Falta de un software dedicado, uso de Excels vulnerables.
 - **Procesos (Métodos):** Flujos manuales, falta de validación de identidad (Token).
 - **Datos:** Información duplicada, falta de integridad en los precios.
 - **Personas:** Dependencia de la memoria de los trabajadores, errores humanos en el tipoo.

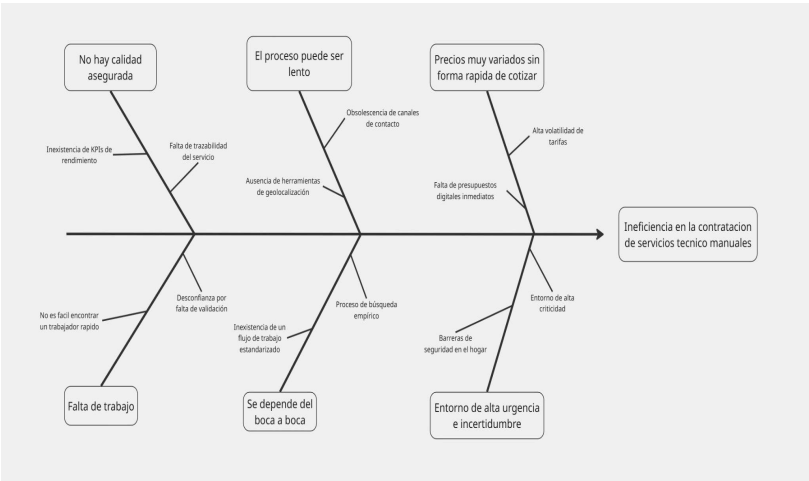


Fig: Diagrama de Árbol.

Link del Diagrama de Árbol [aquí](#).

Al realizar el análisis de Ishikawa, el equipo identificó que la causa raíz no es la falta de personal, sino la carencia de una infraestructura de datos (Capa de Persistencia) que asegure la integridad de los servicios.

2.4 Estado del Arte y Homologación

El desarrollo de software competitivo requiere un análisis comparativo riguroso frente a las soluciones existentes en el ecosistema digital. No basta con identificar la competencia, sino determinar su "techo tecnológico" y las carencias en su propuesta de valor. A continuación, se presenta una matriz de homologación que contrasta a **Contrabajo** con plataformas líderes, evidenciando cómo nuestra arquitectura de geolocalización en tiempo real y validación de identidad mediante el número de documento cubre una brecha crítica en la seguridad y la inmediatez de los servicios técnicos domésticos.

Criterio	Marketplace (Facebook)	Pololitos	Fiverr	Contrabajo
Naturaleza del Servicio	Generalista / Informal.	Oficios rápidos locales.	Digital / Freelance Global.	Técnico / Emergencia Doméstica.
Ubicación del Prestador	Radio aproximado.	Lista por comuna.	Cualquier parte del mundo.	Geolocalización Real (GPS).
Validación de Identidad	Nula.	Nula.	Media (Perfil verificado).	Alta (Número de Documento).
Inmediatez (Urgencia)	Baja (esperar respuesta).	Media (contactar lista).	Nula (entrega en días).	Alta (prestador verificado)
Seguridad en el Hogar	Inexistente.	Media.	No aplica (es remoto).	Máxima (Filtros de confianza).

2.5 Situación Inicial (As-Is)

Para dimensionar el impacto de una solución tecnológica, es necesario analizar el proceso actual de contratación de servicios técnicos, caracterizado por ser mayormente informal y poco estructurado. El modelo "As-Is" (**situación actual**) permite identificar un flujo fragmentado, donde aspectos críticos como la seguridad, los tiempos de respuesta y la transparencia en los costos no se encuentran adecuadamente gestionados.

En este contexto, la contratación de servicios suele depender de recomendaciones informales o plataformas generalistas, lo que introduce incertidumbre tanto para el cliente como para el prestador. A continuación, se presenta un diagrama de flujo que ilustra el proceso actual e identifica sus principales puntos críticos.

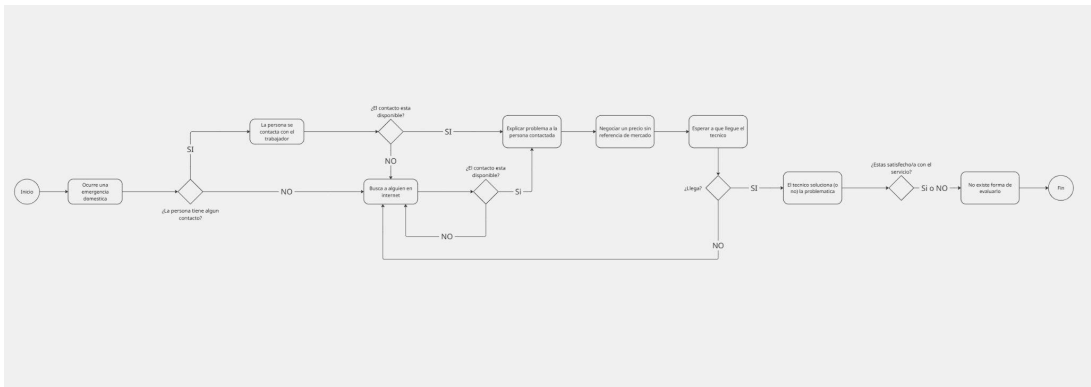


Fig: As-Is.

Link de diagrama [aquí](#).

Análisis de Puntos de Falla en el Proceso "As-Is"

- **Ineficiencia en la búsqueda (cuello de botella):**
El usuario puede invertir entre 30 y 60 minutos en la búsqueda de un prestador adecuado, debido a la dispersión de la oferta y la falta de mecanismos de filtrado efectivos.
- **Riesgos de seguridad:**
No existen mecanismos formales de validación de identidad, lo que implica que el cliente basa su decisión en información limitada o no verificada.
- **Baja transparencia en costos:**
La definición del precio suele realizarse de manera informal, lo que puede generar discrepancias o sobrecostos durante la ejecución del servicio.
- **Ausencia de trazabilidad y postventa:**
Al no existir un registro formal de la transacción, se dificulta la gestión de reclamos, evaluaciones del servicio o seguimiento posterior.

2.6 Justificación del proyecto

La necesidad del proyecto se fundamenta en una brecha estructural entre la oferta y la demanda de servicios técnicos en Chile y Latinoamérica. Diversos estudios evidencian una alta disponibilidad de trabajadores independientes y microemprendedores que, en su mayoría, operan en condiciones de informalidad y con un bajo nivel de digitalización. Esta situación limita su visibilidad en el mercado y reduce su acceso a oportunidades laborales formales.

Por otro lado, los usuarios que requieren este tipo de servicios enfrentan múltiples fricciones durante el proceso de búsqueda y contratación, tales como la ausencia de mecanismos confiables de validación de identidad, la falta de sistemas de reputación y la dificultad para comparar alternativas de manera eficiente. Estas limitaciones generan incertidumbre respecto a la calidad del servicio y a la confiabilidad del prestador.

Adicionalmente, la existencia de herramientas institucionales, como el buscador de instaladores autorizados de la SEC, evidencia la necesidad de contar con sistemas que permitan verificar la idoneidad de los prestadores y acceder a información confiable al momento de contratar servicios técnicos.

A partir del análisis causa-efecto realizado, se concluye que la problemática no radica en la falta de oferta, sino en la ausencia de una infraestructura digital que permita organizar, validar y dar trazabilidad a la interacción entre los actores involucrados.

En este contexto, *Contrabajo* se propone como una solución tecnológica que digitaliza el proceso de intermediación de servicios técnicos, incorporando funcionalidades como geolocalización, validación de identidad, sistemas de reputación y gestión de interacciones entre usuarios. Estas herramientas buscan mejorar la eficiencia del proceso, aumentar la seguridad y fortalecer la confianza en la contratación de servicios.

De esta manera, la solución propuesta no solo responde a una necesidad tecnológica, sino que aborda una problemática estructural del mercado, contribuyendo a la formalización y modernización del sector de servicios técnicos.

2.6.1 Historias de usuario

Para reforzar el análisis realizado, se llevaron a cabo diversas entrevistas a potenciales usuarios. A partir de la información recopilada, se elaboraron mapas de empatía que permitieron identificar necesidades, comportamientos y requerimientos según distintos perfiles de usuario. Con base en estos hallazgos, se definieron las siguientes historias de usuario, las cuales pueden revisarse en mayor detalle en sus respectivas tablas junto con sus criterios de aceptación.

HU-01	Como usuario (cliente o trabajador), quiero registrarme e iniciar sesión de forma segura, para acceder a la plataforma con un perfil propio y confiable.
HU-02	Como trabajador, quiero crear, editar y eliminar publicaciones de mis servicios, para mostrar mi oferta de manera clara y atraer clientes.
HU-03	Como trabajador, quiero agregar descripción, categoría, precio e imagen a cada servicio, para que los clientes entiendan lo que ofrezco sin tener que preguntarme todo por mensaje.
HU-04	Como cliente, quiero visualizar publicaciones en formato claro y ordenado, para comparar varias opciones de servicio rápidamente.
HU-05	Como cliente, quiero acceder al detalle de cada servicio, para revisar información relevante antes de decidir contactar.
HU-06	Como cliente, quiero navegar entre publicaciones desde la vista detalle, para revisar alternativas sin perder tiempo volviendo atrás.
HU-07	Como cliente, quiero buscar servicios por categoría, palabra clave y cercanía, para encontrar opciones adecuadas a mi necesidad y ubicación.
HU-08	Como cliente, quiero filtrar por disponibilidad y zona de atención, para encontrar técnicos que realmente puedan atenderme cuando los necesito.
HU-09	Como cliente, quiero iniciar una conversación con un trabajador desde una publicación, para resolver dudas antes de contratar.
HU-10	Como cliente o trabajador, quiero que el chat quede registrado dentro de la app, para tener respaldo de lo acordado y mayor seguridad.

HU-1 1	Como cliente, quiero generar una cita de servicio desde el chat, para formalizar la atención con fecha y hora definidas.
HU-1 2	Como cliente o trabajador, quiero ver el estado de una cita, para saber si está pendiente, confirmada, en proceso o finalizada.
HU-1 3	Como cliente, quiero calificar un servicio con puntuación y comentario, para compartir mi experiencia y ayudar a otros usuarios a decidir.
HU-1 5	Como trabajador, quiero activar o desactivar la disponibilidad de mis servicios, para aparecer solo cuando realmente puedo atender.
HU-1 6	Como cliente, quiero reportar a un usuario o publicación, para sentirme protegido ante comportamientos indebidos o información engañosa.
HU-1 7	Como moderador, quiero revisar reportes y aplicar medidas sobre cuentas.
HU-1 8	Como usuario, quiero cerrar sesión de forma segura, para proteger mi cuenta cuando dejo de usar la app.
HU-1 9	Como usuario, quiero visualizar y editar mi perfil, para mantener actualizada mi información personal y de contacto.
HU-2 0	Como usuario, quiero verificar mi cuenta, para demostrar que soy una persona real y generar confianza en los demás.
HU-2 1	Como usuario, quiero recuperar mi cuenta de forma segura, para volver a acceder sin perder mis datos ni mi historial.
HU-2 2	Como usuario, quiero recibir y revisar notificaciones, para enterarme de mensajes, cambios de estado y citas pendientes.

Estas historias sirven como base para la definición de **requerimientos funcionales, casos de uso** y flujos de interacción de la aplicación (detallados más adelante en el presente informe), permitiendo mantener coherencia entre el análisis inicial del problema y el desarrollo técnico de la solución. De esta manera, las **historias de usuario** no solo facilitan la comprensión de los objetivos del sistema, sino que también contribuyen a orientar el diseño de una plataforma centrada en las necesidades reales de clientes, trabajadores y moderadores.

3. OBJETIVOS Y ALCANCE (IL 1.2)

La definición de objetivos constituye una etapa fundamental dentro del proceso de desarrollo de software, ya que permite transformar una idea conceptual en un conjunto de metas técnicas claras, medibles y alcanzables. En este sentido, los objetivos no solo orientan el rumbo del proyecto, sino que también establecen los criterios bajo los cuales se evaluará su éxito, asegurando coherencia entre la problemática identificada, la solución propuesta y su implementación tecnológica.

En el contexto del proyecto **Contrabajo**, la formulación de objetivos adquiere especial relevancia debido a la necesidad de abordar una problemática estructural mediante una solución digital. Esto implica no sólo definir qué se desea construir, sino también cómo se construirá, bajo qué estándares y con qué restricciones técnicas. De esta forma, los objetivos permiten alinear decisiones de diseño, tales como la elección de una arquitectura basada en microservicios, el uso de tecnologías específicas y la estructuración de los datos, con los requerimientos funcionales y no funcionales del sistema.

Asimismo, los objetivos sirven como guía para la planificación y ejecución del proyecto, facilitando la organización del trabajo en etapas o iteraciones, y permitiendo un seguimiento efectivo del avance. En entornos de desarrollo ágil, como el adoptado en este proyecto, estos objetivos se traducen en hechos concretos que pueden ser abordados progresivamente, asegurando una entrega incremental de valor y una validación continua de la solución.

A continuación, se presenta el objetivo general del proyecto junto con los objetivos específicos que estructuran su desarrollo, los cuales en conjunto guían la implementación de la solución *Contrabajo* y permiten materializar su propuesta de valor en un sistema funcional, robusto y alineado con las necesidades del mercado identificado.

3.1 Objetivo General

Desarrollar una aplicación móvil que facilite la conexión rápida y eficiente entre clientes y trabajadores especializados (como gasfiteros, soldadores, carpinteros, entre otros), mediante una plataforma tipo marketplace de servicios. La aplicación permitirá a los trabajadores publicar y gestionar múltiples servicios, mientras que los clientes podrán encontrarlos y explorarlos de forma inmediata a través de una interfaz intuitiva, basada en visualización en cuadrícula y navegación fluida entre publicaciones. Además, se incorporarán herramientas de análisis y estadísticas que permitan a los trabajadores optimizar su desempeño dentro de la plataforma. La solución se implementará sobre una arquitectura robusta de microservicios en Java, con un frontend reactivo en Kotlin y una infraestructura escalable en la nube de Microsoft Azure, garantizando alto rendimiento, seguridad y facilidad de mantenimiento.

3.2 Objetivos Específicos

3.2.1 Conexión Cliente–Trabajador

Implementar un sistema que facilite el contacto directo entre clientes y trabajadores a partir de publicaciones de servicios, permitiendo iniciar interacciones mediante mensajería integrada y reduciendo los tiempos de búsqueda y coordinación.

3.2.2 Lógica de Negocio (Backend)

Desarrollar un ecosistema de microservicios en Java (Spring Boot y Spring Cloud) que gestione de manera distribuida los usuarios, publicaciones de servicios, chats de contacto y citas de servicio, asegurando una arquitectura modular, escalable y mantenible.

3.2.3 Seguridad y Confiabilidad

Diseñar e integrar mecanismos de autenticación y verificación de identidad mediante validación de datos, garantizando la confiabilidad de los usuarios y la seguridad en las interacciones dentro de la plataforma.

3.2.4 Arquitectura de Datos

Modelar y estructurar una base de datos relacional en Microsoft SQL Server que permita la gestión consistente de usuarios, publicaciones, conversaciones, citas de servicio y sistemas de reputación, asegurando la integridad, persistencia y trazabilidad de la información.

3.2.5 Experiencia de Usuario (UX/UI)

Desarrollar una interfaz móvil tipo marketplace, basada en visualización en cuadrícula y navegación fluida entre publicaciones, que permita una exploración rápida, simple e intuitiva de los servicios disponibles.

3.2.6 Infraestructura y Despliegue

Implementar y desplegar la solución en una arquitectura de infraestructura como servicio (IaaS) en Microsoft Azure, utilizando contenedores Docker y la herramienta de orquestación Docker Compose para asegurar la portabilidad y la continuidad operativa del sistema.

3.2.7 Gestión Ágil del Proyecto

Administrar el desarrollo del sistema mediante metodología Scrum, utilizando herramientas como Jira para la planificación, seguimiento de tareas y cumplimiento de hitos técnicos y académicos.

3.3 Alcance Técnico

El alcance delimita las fronteras del proyecto, definiendo los compromisos de entrega y las condiciones de contorno bajo las cuales opera el equipo de desarrollo. Esta sección es vital para gestionar las expectativas y asegurar la factibilidad técnica de la solución planteada.

3.3.1 Entregables del Proyecto

- **Software Funcional:** Aplicación móvil instalable para Android (APK) integrada de manera síncrona con el ecosistema de microservicios en Java (Spring Boot) y persistencia de datos en Microsoft SQL Server orquestado en contenedores.
- **Infraestructura Cloud:** Entorno configurado en una Máquina Virtual de Microsoft Azure que soporta el despliegue y la intercomunicación de los microservicios y el motor de base de datos bajo contenedores Docker.
- **Documentación Técnica:**
 - **Especificación de Requerimientos:** Documento detallado de RF y RNF.
 - **Diseño de Arquitectura:** Diagramas UML (Casos de uso, Clases) y Modelo Entidad-Relación (MER).
 - **Diccionario de API:** Documentación de los endpoints desarrollados en Spring Boot.
- **Manuales de Operación:** Guía de usuario para el flujo de cliente/trabajador y manual de despliegue técnico.

3.3.2 Supuestos del Proyecto

- **Conectividad:** Se asume que los usuarios finales disponen de conexión a internet constante para el funcionamiento de los microservicios y la geolocalización.
- **Acceso a APIs de Terceros:** Se considera el uso de servicios externos para la visualización de mapas (Google Maps API o similar) y almacenamiento de imágenes en la nube.
- **Hardware del Usuario:** Los dispositivos móviles cuentan con sensores GPS activos para que el sistema de proximidad funcione correctamente.

3.3.3 Restricciones del Proyecto

- **Presupuesto:** El desarrollo debe ajustarse estrictamente a la "Capa Gratuita" (Free Tier) de Microsoft Azure, lo que limita el escalado vertical de los servidores y el almacenamiento de la base de datos a los recursos gratuitos disponibles.
- **Tiempo de Ejecución:** El proyecto cuenta con un plazo improrrogable de 16 semanas para pasar de la fase de concepción a la entrega del producto funcional al finalizar el semestre académico.
- **Geográfica:** La aplicación estará limitada al territorio nacional para validar la precisión del mecanismo de búsqueda y ordenamiento por proximidad.

4. PROPUESTA DE SOLUCIÓN Y DISEÑO CONCEPTUAL (IL 1.1 / IL 1.4)

4.1 Definición de Requerimientos

Una vez diagnosticada la problemática y establecidos los objetivos del proyecto, se da paso a la fase de diseño conceptual de la solución, en la cual se traducen las necesidades del usuario en requerimientos funcionales y no funcionales del sistema. Esta etapa resulta clave, ya que permite definir de manera estructurada el comportamiento esperado de la aplicación, así como las interacciones entre los distintos actores involucrados.

En este contexto, los requerimientos se modelan mediante **Casos de Uso**, los cuales representan las distintas funcionalidades que el sistema debe ofrecer, desde la perspectiva de los usuarios. Este enfoque permite identificar de forma clara las acciones que pueden realizar tanto los clientes como los prestadores de servicios, así como los flujos de interacción asociados a procesos como el registro de usuarios, la publicación de servicios, la comunicación entre actores y la coordinación de ofertas.

Cada caso de uso ha sido diseñado considerando un enfoque modular, alineado con la arquitectura basada en microservicios definida para la solución. Esto implica que las funcionalidades del sistema se distribuyen en componentes independientes, tales como gestión de usuarios, servicios, comunicaciones y soporte, permitiendo una mayor escalabilidad, mantenibilidad y desacoplamiento entre las distintas capas del sistema.

De esta manera, la definición de requerimientos no solo establece qué funcionalidades debe cumplir el sistema, sino que también sienta las bases para el diseño estructural de la aplicación, permitiendo una transición coherente hacia el modelado de datos (MER) y la arquitectura técnica que sustenta la plataforma **Contrabajo**.

Antes de definir los casos de uso específicos del sistema, resulta necesario identificar los actores que participarán en las distintas interacciones dentro de la plataforma. Esta etapa permite determinar qué tipos de usuarios utilizarán el sistema, cuáles serán sus responsabilidades y qué funcionalidades requerirán para cumplir sus objetivos dentro de la aplicación.

4.1.1 Identificación de Actores

Para la definición de los casos de uso del sistema, es necesario identificar los actores que interactúan con la plataforma. En este contexto, un actor se define como cualquier entidad externa que se comunica con el sistema y participa en alguno de sus procesos, ya sea ejecutando acciones o recibiendo información.

En el sistema *Contrabajo*, se identifican los siguientes actores principales:

- **Cliente:**
Corresponde al usuario que demanda servicios técnicos dentro de la plataforma. Sus principales interacciones incluyen la búsqueda y exploración de servicios publicados, la aplicación de filtros para acotar resultados, el inicio de conversaciones con prestadores, la coordinación de citas y la evaluación posterior de los servicios recibidos. Este actor representa el lado de la demanda dentro del sistema.
- **Trabajador (Prestador de Servicio):**
Corresponde al usuario que ofrece servicios a través de la plataforma. Es responsable de crear y gestionar sus publicaciones, definir condiciones como precio o disponibilidad, responder solicitudes de contacto y ejecutar los servicios acordados. Este actor representa el lado de la oferta y es clave en la generación de contenido dentro del sistema.
- **Moderador:**
Corresponde a un actor con funciones de supervisión y control dentro de la plataforma. Su rol está orientado a garantizar el cumplimiento de las normas de uso, mediante la revisión de reportes, la detección de comportamientos indebidos y la aplicación de medidas correctivas, tales como restricciones o bloqueos de usuarios o contenido.

Es importante destacar que, a nivel de modelamiento, tanto el cliente como el trabajador corresponden a una misma entidad base dentro del sistema (usuario), diferenciándose a través de atributos como el tipo de perfil. Esto permite mantener un diseño flexible y escalable, evitando la duplicación de estructuras de datos y facilitando la gestión de roles dentro de la plataforma.

Asimismo, ciertos usuarios pueden acceder a funcionalidades adicionales según su nivel o tipo de cuenta (por ejemplo, acceso a métricas o estadísticas), lo cual se gestiona a nivel de permisos o atributos del sistema, sin implicar la definición de nuevos actores.

4.1.2 Casos de uso

Tras la definición de los perfiles que convergen en el ecosistema de **Contrabajo**, se procede a la especificación de los **Casos de Uso (CU)**.

Estas representaciones constituyen el puente entre las necesidades del negocio y la implementación técnica, estableciendo los requisitos funcionales que la arquitectura de microservicios debe satisfacer.

Cada interacción ha sido diseñada para garantizar una navegación fluida en el frontend (Kotlin) y una persistencia de datos consistente en el backend (Java), asegurando que la comunicación se ejecute de forma íntegra y segura.

A continuación, se presenta la matriz de interacciones principales que rigen el comportamiento del sistema:

ID	Actor	Caso de Uso	Descripción Técnica
CU-1	Cliente	Búsqueda y Filtro	Localizar técnicos por categoría, palabra clave y radio de distancia (GPS).
CU-2	Cliente	Explorar Publicaciones	Visualizar publicaciones de servicios en formato cuadrícula y navegar de forma lateral entre publicaciones desde la vista detalle para revisar opciones de manera rápida.
CU-3	Cliente	Reportes	Denunciar publicaciones o prestadores por mal comportamiento, incumplimiento o información engañosa.
CU-4	Trabajador	Ofrecer Servicios	Crear y administrar una o más publicaciones de servicios, incluyendo descripción, imágenes, precios y condiciones de atención.

CU-5	Trabajador	Estado de Disponibilidad	Switch de disponibilidad en cada servicio ofrecido para aparecer o no en las búsquedas.
CU-6	Trabajador	Dashboard de Estadísticas	Visualizar estadísticas y métricas de visualizaciones de publicaciones, contactos iniciados y citas de servicio. Funcionalidad disponible según tipo de cuenta.
CU-7	Moderador	Gestión de Comunidad	Revisar reportes y ejecutar "Baneo" (suspensión) de cuentas.
CU-8	Cliente/ Trabajador	Chat de Contacto	Generar una conversación directa entre cliente y trabajador a partir de una publicación de servicio, sin exponer de inmediato datos personales sensibles.
CU-9	Cliente/ Trabajador	Cita de Servicio	Formalizar una solicitud o coordinación dentro del chat de contacto, registrando fecha, estado y datos básicos de la atención.

Es importante destacar que estos casos de uso no operan de forma aislada, sino que se encuentran integrados con distintos componentes de la arquitectura basada en microservicios implementada sobre **Azure**. Cada funcionalidad interactúa con servicios específicos según su propósito dentro de la plataforma. Por ejemplo, el caso de uso **CU-1** consume servicios de geolocalización para realizar búsquedas por proximidad, mientras que los casos asociados a comunicación y coordinación de servicios permiten mantener trazabilidad sobre las interacciones realizadas entre clientes y trabajadores.

La definición de los casos de uso permitió establecer de manera estructurada las principales funcionalidades requeridas por la plataforma Contrabajo, considerando las necesidades e interacciones de cada actor identificado dentro del sistema. A través de este modelamiento fue posible delimitar responsabilidades, flujos de interacción y procesos clave asociados a la búsqueda, oferta y coordinación de servicios técnicos.

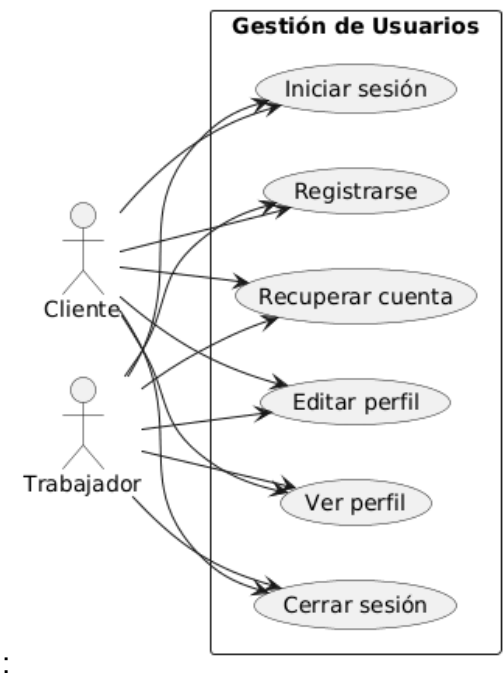
4.1.3 Diagramas de Casos de Uso

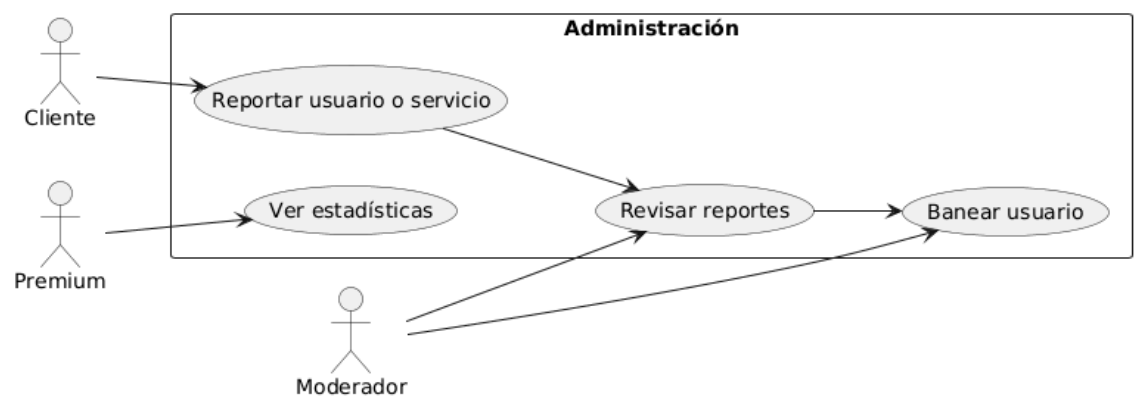
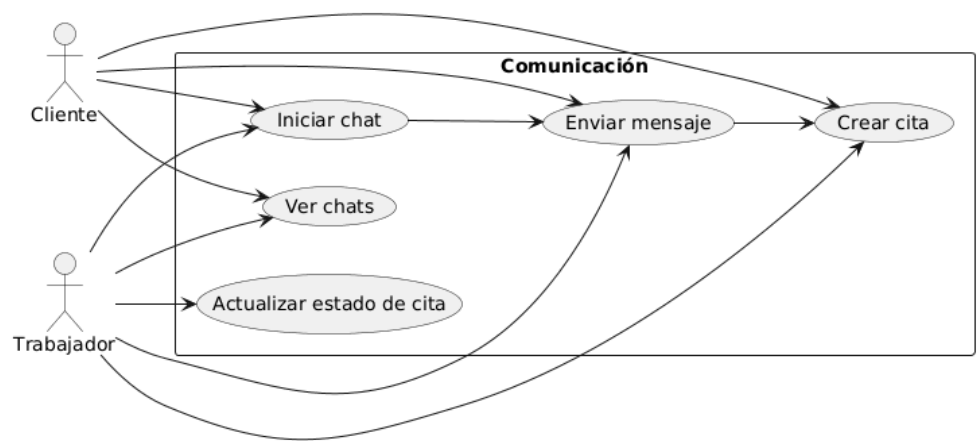
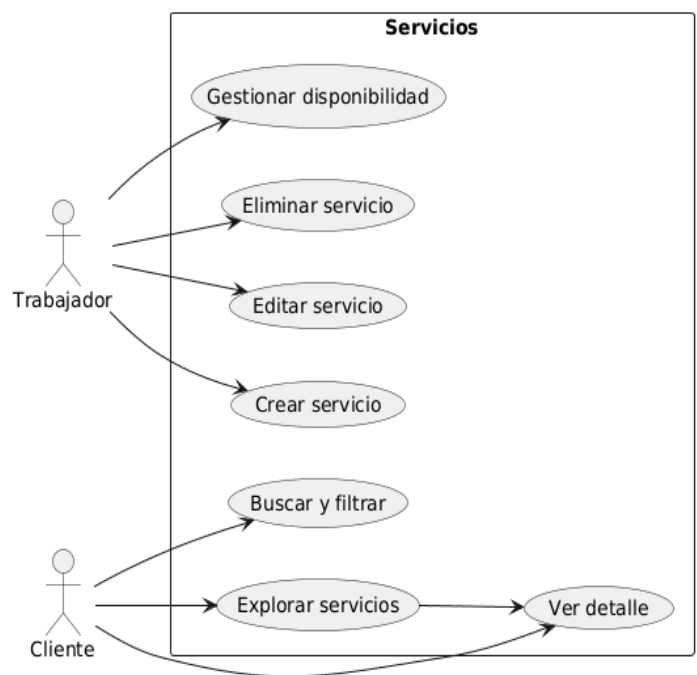
Los Diagramas de Casos de Uso, pertenecientes al lenguaje de modelado UML (Unified Modeling Language), permiten representar de manera gráfica las funcionalidades principales de un sistema desde la perspectiva de los usuarios que interactúan con él. Su objetivo es describir qué acciones puede realizar cada actor dentro de la plataforma y cómo se relaciona con los distintos procesos definidos en el sistema, sin profundizar aún en detalles técnicos de implementación.

Si bien, más adelante se abordará en mayor profundidad dicho concepto, es necesario explicar que la representación gráfica de los requerimientos funcionales mediante **Diagramas de Casos de Uso** permite visualizar de manera clara las interacciones entre los actores y el sistema, facilitando la comprensión del comportamiento general de la plataforma y el alcance de sus funcionalidades. En el contexto de **Contrabajo**, estos diagramas permiten modelar los procesos asociados a la búsqueda, publicación y coordinación de servicios técnicos, asegurando que las interacciones entre usuarios se desarrollen de forma organizada, segura y coherente con las reglas de negocio definidas.

Asimismo, los diagramas constituyen una guía de referencia para el desarrollo e implementación del sistema, permitiendo mantener consistencia entre el diseño conceptual y la arquitectura técnica basada en microservicios. De esta manera, las funcionalidades definidas para la plataforma pueden ser distribuidas en componentes independientes desarrollados mediante tecnologías como Kotlin para el frontend y Java para los servicios backend.

A continuación, se presentan las representaciones gráficas de los principales casos de uso del sistema, organizadas según su propósito funcional y administrativo.





Como se observa en los diagramas, el sistema presenta una arquitectura desacoplada, donde cada microservicio administra funcionalidades y procesos específicos de manera independiente. Esta estructura favorece la escalabilidad, mantenibilidad y organización de la solución, permitiendo además mantener un control adecuado sobre la comunicación entre componentes y la trazabilidad de las solicitudes realizadas dentro de la plataforma.

4.1.4 Modelo Entidad-Relación (MER)

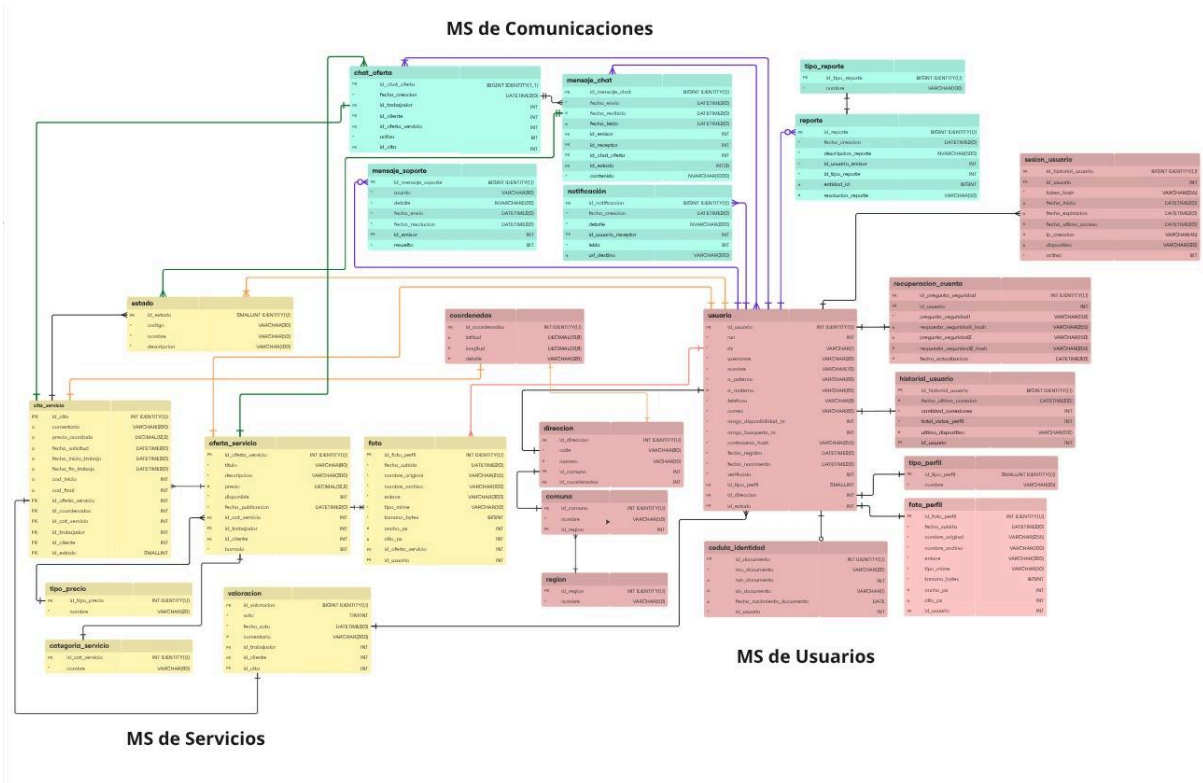
Con el objetivo de estructurar de manera eficiente la información utilizada por la plataforma Contrabajo, se diseñó un Modelo Entidad-Relación (MER) que representa las entidades principales del sistema, sus atributos y las relaciones existentes entre ellas.

El modelo de datos fue construido considerando los requerimientos funcionales definidos previamente y la arquitectura basada en microservicios implementada en el proyecto. Su propósito es garantizar integridad, consistencia y trazabilidad de la información, permitiendo una correcta separación de responsabilidades entre los distintos módulos del sistema.

La estructura general del modelo se encuentra dividida en tres grandes dominios funcionales, cada uno correspondiente directamente a un microservicio independiente dentro de la arquitectura de la solución:

- Gestión de usuarios.
- Gestión de servicios.
- Gestión de comunicaciones.

A continuación, se presenta el Modelo Entidad-Relación correspondiente al sistema:



Link del MER [aquí](#).

Descripción General del Modelo

Dominio de Usuarios

Este dominio constituye el núcleo de identidad de la plataforma. Está encabezado por la entidad **usuario** (o **cuenta_usuario**), la cual centraliza las credenciales de acceso, correos electrónicos, contraseñas encriptadas y el estado operativo del perfil (activo/inactivo) requerido para los filtros de seguridad. Directamente vinculada a esta estructura se encuentra la entidad **rol** (o **perfil**), la cual implementa el control de acceso basado en roles (RBAC) para discriminar los permisos específicos de "Cliente", "Trabajador" o "Administrador" dentro del ecosistema. Complementariamente, se incorporan entidades destinadas a almacenar los datos de perfil específicos, información de contacto, valoraciones promedio y, de ser necesario, la entidad **coordenadas** (o **ubicacion_usuario**), indispensable para alimentar los algoritmos de búsqueda y ordenamiento de servicios por proximidad geográfica.

Dominio de servicios

La entidad **oferta_servicio** representa las publicaciones realizadas por los trabajadores, permitiendo almacenar información como título, descripción, categoría, disponibilidad y tipo de precio del servicio ofrecido. Asociada a esta entidad se encuentra la tabla **foto**, la cual administra las imágenes vinculadas a las publicaciones, incluyendo campos para el tipo MIME, dimensiones y rutas de almacenamiento, permitiendo así una gestión multimedia desacoplada. Por otra parte, la entidad **cita_servicio** permite formalizar los acuerdos entre cliente y trabajador, registrando fechas de atención, estado del servicio, precio acordado y ubicación, actuando como el eje de trazabilidad operacional dentro del proceso de contratación. Además, la entidad **valoracion** permite registrar las evaluaciones realizadas por los clientes respecto al servicio recibido, contribuyendo a generar transparencia y confianza en la plataforma.

Dominio de comunicaciones

La entidad **chat_oferta** permite gestionar conversaciones privadas entre cliente y trabajador asociadas a una oferta de servicio específica. El sistema incorpora una referencia lógica opcional desde este chat hacia **cita_servicio**, vinculando las conversaciones con los servicios efectivamente coordinados para mantener la trazabilidad entre la interacción inicial y la ejecución del trabajo. Cada conversación contiene múltiples registros en la entidad **mensaje_chat**, persistiendo el historial completo de la interacción. Complementariamente, se incorporan entidades orientadas a soporte, notificaciones y moderación, tales como **mensaje_soporte**, **notificacion** y **reporte**, destinadas a la resolución de incidencias o denuncias realizadas por los usuarios.

Relaciones Principales del Modelo

Las principales relaciones definidas dentro del modelo son las siguientes:

- Un usuario puede publicar múltiples ofertas de servicios.
- Una oferta de servicio pertenece a un único trabajador.
- Una oferta de servicio puede contener múltiples fotografías asociadas.
- Un cliente puede iniciar múltiples conversaciones asociadas a distintas ofertas.
- Cada chat puede contener múltiples mensajes.
- Una conversación puede vincularse opcionalmente a una cita de servicio.
- Una oferta de servicio puede derivar en múltiples citas.
- Una cita puede asociarse a una valoración realizada por el cliente.
- Los usuarios pueden generar reportes asociados a incidencias dentro de la plataforma.

Importancia del Modelo de Datos:

La construcción del MER permitió establecer una base sólida para la implementación física de la base de datos en Microsoft SQL Server, facilitando el desarrollo backend y asegurando la coherencia del sistema.

A diferencia de una arquitectura monolítica tradicional, donde todas las tablas coexisten dentro de una misma base de datos compartiendo claves foráneas físicas (FOREIGN KEY), en esta arquitectura de microservicios se busca reducir el acoplamiento. Para optimizar los recursos de infraestructura en el entorno cloud, los dominios comparten el mismo motor físico de base de datos en Azure, pero se administran bajo un estricto **aislamiento lógico** a nivel de aplicación.

Por ello, las relaciones entre entidades pertenecientes a distintos microservicios no se implementan mediante restricciones físicas en el motor de base de datos, sino mediante **referencias lógicas** basadas en identificadores compartidos. Por ejemplo, entidades del dominio de servicios (oferta_servicio o cita_servicio) almacenan los campos id_trabajador e id_cliente de forma numérica pura, sin establecer una restricción física directa contra la tabla de usuarios, ya que esta pertenece a un dominio ajeno administrado de forma exclusiva por el microservicio de usuarios. De manera similar, la tabla chat_oferta en el dominio de comunicaciones mantiene referencias lógicas mediante los campos id_oferta_servicio e id_cita para relacionar conversaciones con publicaciones y contratos sin generar dependencias estructurales rígidas.

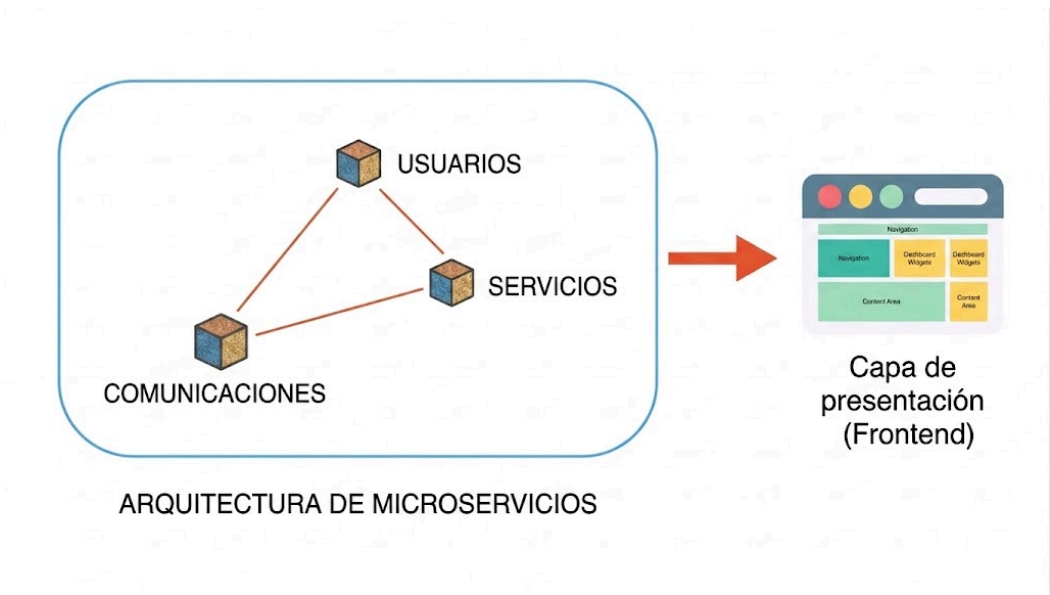
Esta estrategia de referencias lógicas facilita la distribución de los servicios en el entorno cloud. Dado que no existen amarres físicos en la base de datos, cada microservicio puede migrarse, replicarse o cambiarse de servidor de forma elástica a través de Docker Compose sin comprometer la integridad de los demás componentes.

Desacoplamiento y Validación entre Componentes

El uso de referencias lógicas introduce el desafío de asegurar la integridad referencial desde las capas superiores del software. Al no delegar las restricciones en el motor de base de datos de Microsoft SQL Server, la validación de la existencia y validez de los datos inter-dominio debe ser asumida por la lógica de negocio implementada en las APIs de Spring Boot de cada microservicio.

Para resolver este desafío e impedir estados inconsistentes, el ecosistema implementa un mecanismo de **comunicación síncrona y filtros de interceptación** que operan bajo el siguiente flujo técnico:

1. **Interceptación de Peticiones:** Cuando el cliente móvil (Android) realiza una solicitud hacia endpoints críticos del microservicio de **Servicios** o **Comunicaciones** enviando su token de autenticación (JWT), la petición es capturada por un componente de seguridad interno (JwtAuthenticationFilter).
2. **Validación de Identidad Local:** El microservicio receptor decodifica el token y valida criptográficamente su firma utilizando la clave compartida del sistema.
3. **Validación de Estado Remoto (Desacoplada):** Una vez verificada la firma matemática, la API del microservicio emite una consulta HTTP interna dirigida hacia el microservicio de **Usuarios** (utilizando la resolución de nombres nativa de la red de Docker Compose: `http://contrabajo-usuarios-api:8081`). Este endpoint interroga a su respectivo dominio para comprobar si el identificador extraído del token existe realmente y cuenta con un estado de cuenta activo.
4. **Garantía de Integridad:** Si el microservicio de usuarios confirma la validez del registro, el componente autoriza la persistencia del servicio, cita o mensaje solicitado. En caso contrario, se bloquea la operación y se retorna un código de estado de seguridad (403 Forbidden), protegiendo la integridad referencial y operacional del sistema completo de manera 100% desacoplada.

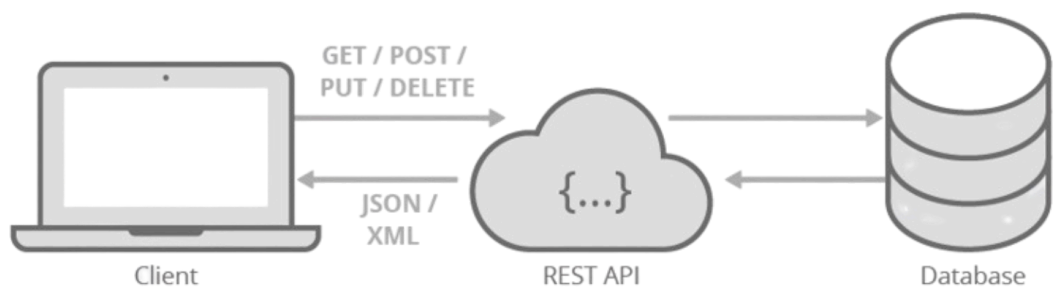


¿Qué permite esta arquitectura?

Esta estrategia arquitectónica permite que cada microservicio pueda evolucionar de manera independiente, desplegarse por separado y escalar según sus propias necesidades de carga. Además, evita que modificaciones estructurales realizadas en un servicio afecten directamente al resto de componentes del sistema, favoreciendo una arquitectura más flexible, mantenible y alineada con las buenas prácticas modernas de desarrollo distribuido.

Otro aspecto relevante es que el desacoplamiento facilita la futura distribución de los servicios en distintos entornos cloud o servidores independientes. Dado que no existen dependencias físicas entre bases de datos, cada microservicio puede migrarse, replicarse o escalarse horizontalmente sin comprometer la integridad estructural del sistema completo. Esto resulta especialmente importante considerando que Contrabajo fue concebido para ejecutarse sobre infraestructura cloud en Microsoft Azure mediante **orquestación con Docker Compose**, donde la portabilidad y la independencia de despliegue representan características fundamentales.

Sin embargo, esta estrategia también introduce nuevos desafíos técnicos. Al no existir claves foráneas físicas entre microservicios, la validación de integridad referencial debe realizarse desde la lógica de negocio implementada en las APIs y servicios internos. Esto implica que, antes de registrar operaciones relacionadas con usuarios, ofertas o citas, el sistema debe verificar mediante comunicación entre microservicios que los identificadores involucrados existan y se encuentren válidos dentro de sus respectivos dominios funcionales.



Finalmente, este modelo de desacoplamiento también permite una mejor separación de responsabilidades dentro del sistema. Mientras el microservicio de usuarios se encarga de la autenticación, perfiles y datos personales, el microservicio de servicios administra publicaciones, citas y valoraciones, y el microservicio de comunicaciones gestiona chats, reportes, soporte y notificaciones. Esta división favorece una organización más clara del sistema y simplifica tanto el desarrollo como la mantención futura de la plataforma.

4.1.6 Diagramas UML

4.1.6.1 Introducción a los Diagramas UML

Con el objetivo de representar de manera estructurada la arquitectura, organización y comportamiento interno del sistema Contrabajo, se utilizaron distintos diagramas UML (Unified Modeling Language), permitiendo modelar tanto los aspectos estáticos como dinámicos de la solución desarrollada.

El uso de UML facilita la documentación técnica del sistema, proporcionando una representación visual estandarizada de los distintos componentes de software, sus responsabilidades, relaciones y flujos de interacción. Esto permite mejorar la comprensión global de la arquitectura, apoyar las tareas de mantenimiento y servir como referencia para futuras etapas de evolución del proyecto.

La construcción de los diagramas se realizó siguiendo principios alineados con modelos modernos de documentación arquitectónica, particularmente el enfoque de vistas múltiples propuesto por el modelo 4+1. Este enfoque establece que una arquitectura de software puede describirse desde distintas perspectivas complementarias, permitiendo representar tanto la estructura lógica del sistema como sus procesos internos, organización física y distribución de componentes.

Dentro de este contexto, los diagramas desarrollados para Contrabajo permiten representar:

- La organización modular de los microservicios y sus capas internas mediante diagramas de paquetes.
- El flujo dinámico de operaciones críticas del sistema a través de diagramas de secuencia.
- La distribución física de los componentes desplegados sobre infraestructura cloud utilizando diagramas de despliegue.

En conjunto, estos diagramas permiten obtener una visión integral de la solución, facilitando tanto el análisis técnico como la validación de las decisiones arquitectónicas implementadas durante el desarrollo del proyecto.

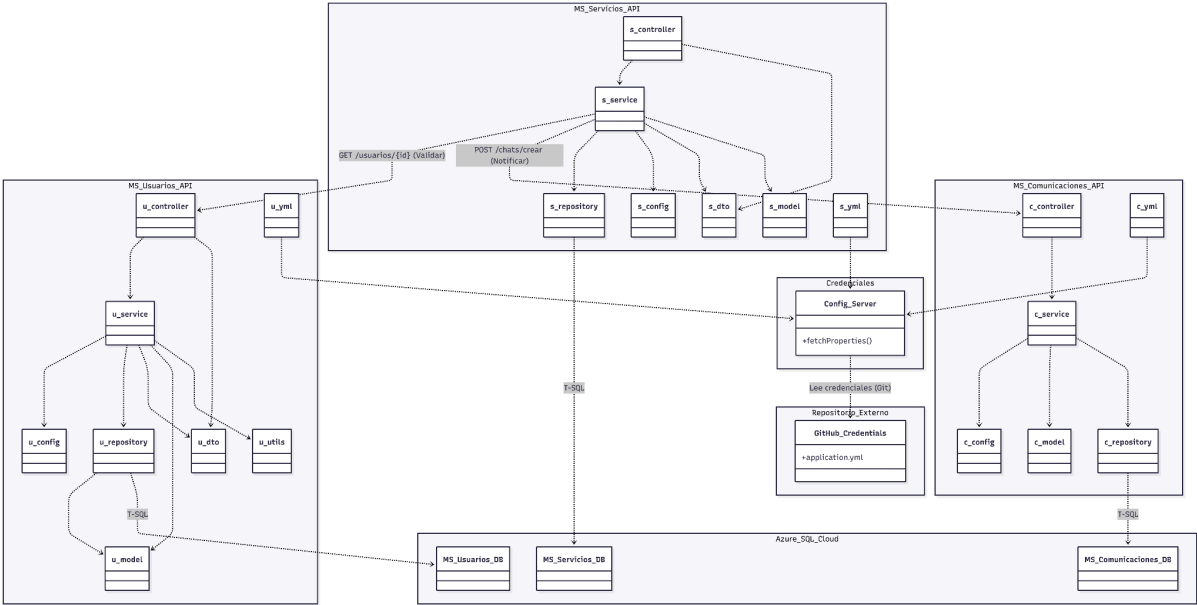
A continuación, se presentan los distintos diagramas UML desarrollados para la plataforma Contrabajo, organizados según la perspectiva arquitectónica y funcional que representan. Cada uno de ellos permite visualizar aspectos específicos del sistema, contribuyendo a comprender la estructura interna de los microservicios, las interacciones entre componentes y la forma en que la solución se despliega sobre la infraestructura tecnológica definida para el proyecto.

4.1.6.2 Diagrama de Paquetes

El diagrama de paquetes permite representar la organización modular del sistema y las dependencias existentes entre los distintos componentes de software que conforman la solución. Este tipo de diagrama resulta especialmente útil en arquitecturas basadas en microservicios, ya que facilita la visualización de la separación lógica entre dominios funcionales, la distribución de responsabilidades y las relaciones de dependencia existentes entre los distintos módulos de la aplicación.

En Contrabajo, el sistema se encuentra estructurado en múltiples microservicios independientes, donde cada componente encapsula un conjunto específico de funcionalidades relacionadas con un dominio funcional determinado, como usuarios, servicios y comunicaciones. Esta separación permite mantener independencia entre los distintos módulos del sistema, reduciendo el acoplamiento y favoreciendo una arquitectura más mantenible y escalable.

Esta organización modular permite mejorar la mantenibilidad del sistema, facilitar futuras modificaciones y reducir el impacto de cambios internos sobre otros componentes de la solución. Asimismo, favorece la reutilización de código, simplifica las tareas de testing y permite que cada microservicio pueda evolucionar de manera independiente según las necesidades del proyecto.

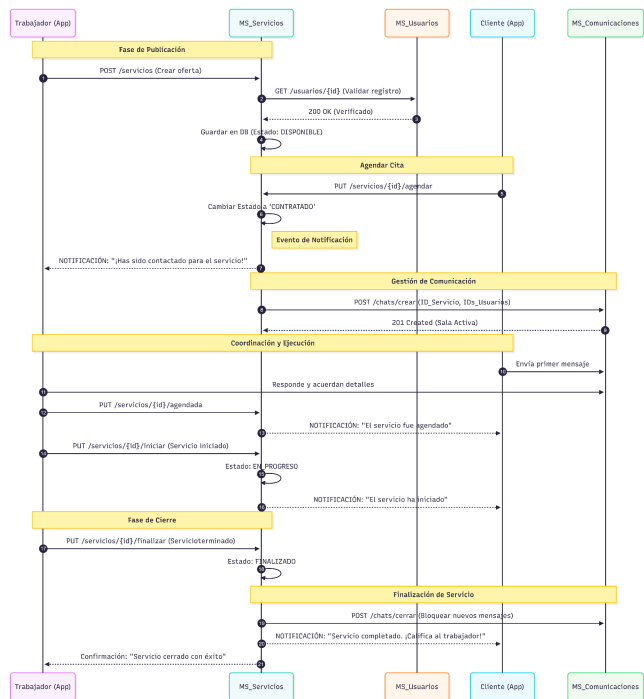


4.1.6.3 Diagrama de Secuencia

El diagrama de secuencia permite representar el comportamiento dinámico del sistema, modelando la interacción temporal entre actores, componentes y servicios durante la ejecución de un proceso específico. Este tipo de diagrama resulta especialmente útil para comprender el flujo de operaciones, las validaciones realizadas por el sistema y la comunicación existente entre las distintas capas de la arquitectura.

En Contrabajo, el sistema se encuentra estructurado en múltiples microservicios independientes, donde cada uno encapsula un conjunto específico de responsabilidades relacionadas con un dominio funcional determinado. Esta separación permite que componentes como gestión de usuarios, servicios y comunicaciones puedan evolucionar de manera autónoma, manteniendo independencia tanto a nivel de lógica de negocio como de persistencia de datos.

Adicionalmente, cada microservicio fue organizado internamente utilizando una arquitectura por capas, siguiendo principios de separación de responsabilidades ampliamente utilizados en el desarrollo de aplicaciones empresariales modernas con Spring Boot. Bajo esta estructura, la capa controller se encarga de gestionar las solicitudes HTTP y exponer los endpoints REST; la capa service concentra la lógica de negocio y las validaciones internas; la capa repository abstrae el acceso a datos mediante Spring Data JPA; mientras que las capas model/entity y dto permiten representar tanto las entidades persistentes como los objetos de transferencia utilizados por las APIs.



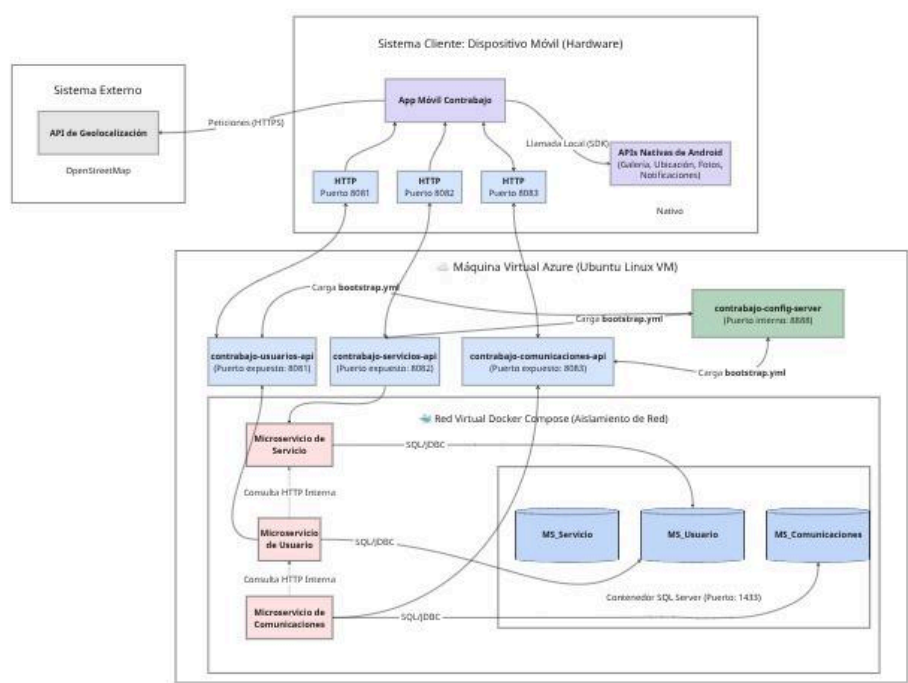
4.1.6.4 Diagrama de Despliegue

El diagrama de despliegue permite representar la distribución física de los componentes del sistema sobre la infraestructura tecnológica utilizada para su ejecución. Este tipo de diagrama modela los nodos de procesamiento, servicios cloud, bases de datos y canales de comunicación existentes entre los distintos elementos desplegados, proporcionando una visión clara de cómo la solución opera a nivel de infraestructura.

En arquitecturas basadas en microservicios, este tipo de representación resulta especialmente importante, ya que permite visualizar la separación física entre componentes, la distribución de responsabilidades y las conexiones existentes entre servicios, bases de datos y clientes externos. Además, facilita la comprensión de aspectos relacionados con escalabilidad, disponibilidad y administración de recursos dentro del entorno de ejecución.

En Contrabajo, el despliegue fue diseñado considerando un entorno cloud basado en Microsoft Azure, donde cada microservicio puede ejecutarse de manera independiente sobre instancias separadas, permitiendo escalabilidad horizontal, aislamiento de fallos y mayor flexibilidad operativa. Esta estrategia permite que un incremento de carga en un componente específico no afecte directamente al resto de servicios del sistema, favoreciendo una arquitectura más resiliente y mantenible.

Asimismo, cada dominio funcional mantiene su propia base de datos independiente en Azure SQL, reforzando el desacoplamiento estructural de la solución y evitando dependencias directas entre microservicios. De esta manera, los servicios relacionados con usuarios, servicios y comunicaciones administran sus datos de forma autónoma, manteniendo independencia tanto a nivel lógico como físico.



4.2 Requisitos Funcionales (RF) - Reglas de Negocio

Los requerimientos funcionales representan las capacidades y comportamientos que el sistema debe implementar para satisfacer las necesidades identificadas durante el levantamiento de información y análisis de necesidades de los usuarios.

ID	Requerimiento Funcional	Descripción
RF-01	Registro e Inicio de Sesión	El sistema debe permitir a los usuarios registrarse e iniciar sesión mediante credenciales seguras.
RF-02	Gestión de Publicaciones	El sistema debe permitir a los trabajadores crear, editar y eliminar publicaciones de servicios, incluyendo información como descripción, categoría, precio e imágenes.
RF-03	Exploración de Servicios	El sistema debe permitir a los clientes visualizar publicaciones en formato cuadrícula y acceder al detalle de cada servicio.
RF-04	Navegación entre Publicaciones	El sistema debe permitir la navegación lateral entre publicaciones desde la vista de detalle, facilitando la exploración continua de servicios.
RF-05	Búsqueda y Filtros	El sistema debe permitir buscar servicios por categoría, palabra clave y proximidad geográfica.
RF-06	Chat de Contacto	El sistema debe permitir iniciar una conversación entre cliente y trabajador a partir de una publicación.
RF-07	Cita de Servicio	El sistema debe permitir generar una cita de servicio dentro del chat, registrando estado y datos básicos de la atención.
RF-08	Sistema de Valoración	El sistema debe permitir a los clientes calificar servicios mediante puntuaciones y comentarios.
RF-09	Gestión de Disponibilidad	El sistema debe permitir al trabajador activar o desactivar la disponibilidad de sus servicios.
RF-10	Reportes y Moderación	El sistema debe permitir reportar usuarios o publicaciones, y habilitar su revisión por parte de un moderador.

4.3 Requisitos No Funcionales (RNF) - Atributos de Calidad

Mientras que los requerimientos funcionales definen *qué* hace el sistema, los **Requisitos No Funcionales (RNF)** determinan *cómo* se comporta la solución ante condiciones operativas específicas.

Para el proyecto **Contrabajo**, estos atributos de calidad son críticos, ya que garantizan que la vinculación entre usuarios se realice bajo un entorno de alta disponibilidad, integridad de datos y eficiencia técnica.

ID	Requerimiento No Funcional	Descripción
RNF-01	Escalabilidad	El sistema debe soportar el crecimiento progresivo de usuarios y publicaciones sin afectar el rendimiento, gracias a una arquitectura basada en microservicios.
RNF-02	Rendimiento	Las operaciones de carga de publicaciones y navegación deben ejecutarse de forma fluida, con tiempos de respuesta adecuados para una experiencia móvil óptima.
RNF-03	Seguridad	El sistema debe proteger la información sensible de los usuarios mediante autenticación segura y control de acceso a los datos.
RNF-04	Disponibilidad	La plataforma debe estar disponible de forma continua, permitiendo el acceso a funcionalidades críticas como búsqueda y contacto en cualquier momento.
RNF-05	Usabilidad	La interfaz debe ser intuitiva y permitir que un usuario pueda encontrar y contactar un servicio en pocos pasos.
RNF-06	Compatibilidad	La aplicación debe funcionar correctamente en dispositivos Android dentro de un rango estándar de versiones del sistema operativo.
RNF-07	Integridad de Datos	La información almacenada en la base de datos debe mantenerse consistente, evitando duplicidad o pérdida de registros.

4.4 Diseño de Interfaz (Mockups)

El diseño visual de Contrabajo materializa una propuesta de valor basada en rapidez, claridad y facilidad de uso. A continuación, se presenta un prototipo de baja/media fidelidad que ilustra una interfaz tipo marketplace, donde el usuario puede explorar publicaciones de servicios en formato cuadrícula y, al acceder al detalle de una publicación, navegar horizontalmente entre otras opciones para agilizar la revisión de servicios disponibles.

4.4.1 Contrabajo Prototipo v1

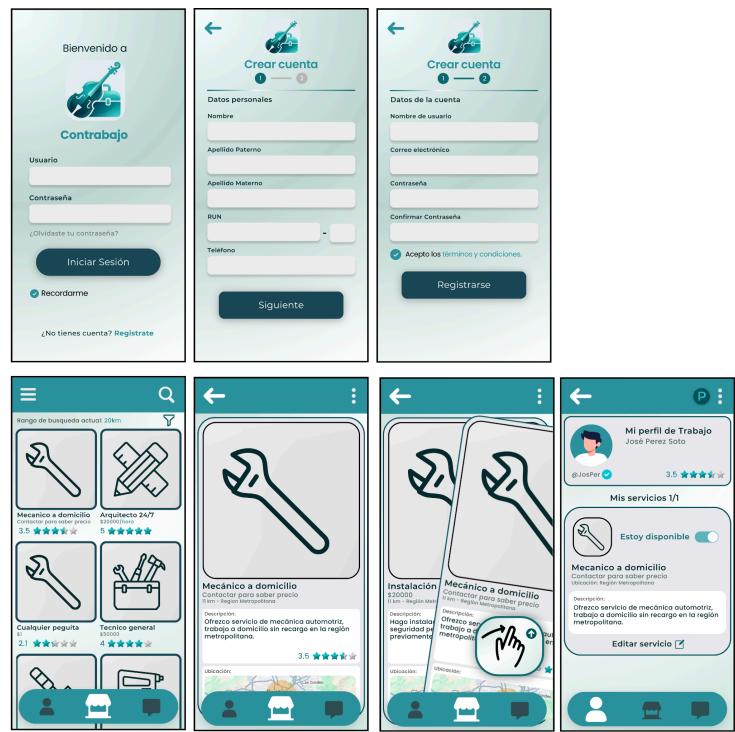


Fig: Mockup Contrabajo

Link del mockup [aquí](#).

Como se observa en el prototipo desarrollado, el diseño de interfaz de **Contrabajo** fue concebido bajo criterios de usabilidad, navegación intuitiva y organización modular de funcionalidades. La estructura visual tipo marketplace permite que los usuarios identifiquen rápidamente servicios disponibles mediante una distribución clara de publicaciones, facilitando la exploración continua y reduciendo la fricción durante el proceso de búsqueda y contacto.

Además, el prototipo refleja la integración entre los distintos módulos funcionales del sistema, tales como autenticación, gestión de servicios, mensajería, perfil y moderación, evidenciando una organización coherente con el enfoque de microservicios utilizado en el backend de la plataforma. De esta manera, el diseño de interfaz no solo cumple una función estética, sino que también actúa como una representación funcional de la arquitectura y flujos operativos definidos para **Contrabajo**.

4.5 Arquitectura y Patrones

La solución fue diseñada bajo un enfoque de separación de responsabilidades, utilizando arquitecturas y patrones de diseño orientados a mejorar la organización interna del sistema, facilitar el mantenimiento del código y permitir la evolución progresiva de la plataforma tanto en el entorno móvil como en los servicios backend.

4.5.1 FrontEnd: Arquitectura MVVM (Model-View-ViewModel)

Para el desarrollo de la aplicación móvil se seleccionó el patrón arquitectónico **MVVM** (Model-View-ViewModel), recomendado oficialmente por Google para aplicaciones Android modernas desarrolladas en **Kotlin** y **Jetpack Compose**.

Este patrón permite separar la interfaz gráfica de la lógica de negocio y del manejo de estados. Dentro de esta arquitectura, el **ViewModel** actúa como intermediario entre la vista y las fuentes de datos, administrando el estado de la aplicación durante el ciclo de vida de Android.

Asimismo, **MVVM** favorece la modularidad y mantenibilidad del frontend, facilitando la reutilización de componentes y las pruebas unitarias de la lógica de presentación de forma independiente a la interfaz gráfica.

4.5.2 BackEnd: Microservicios y Patrón de Capas

El backend de **Contrabajo** fue diseñado bajo una arquitectura basada en microservicios, permitiendo dividir las principales responsabilidades del sistema en servicios independientes orientados a dominios funcionales específicos, tales como autenticación, gestión de servicios y comunicaciones.

Cada microservicio implementa una **arquitectura por capas**, separando la lógica de presentación, negocio y acceso a datos para mantener una estructura organizada y mantenible.

Asimismo, el enfoque basado en microservicios favorece el desacoplamiento entre componentes, permitiendo que cada módulo pueda desarrollarse, mantenerse y desplegarse de manera independiente, facilitando futuras estrategias de escalabilidad y despliegue cloud.



4.6 Estrategia Cloud

La estrategia de infraestructura para Contrabajo fue definida considerando criterios de compatibilidad tecnológica, flexibilidad en la configuración, aislamiento de componentes y viabilidad presupuestaria dentro de los plazos establecidos para el proyecto. En este contexto, se optó por una solución cloud híbrida basada en contenedores que permite centralizar el ecosistema de microservicios y la persistencia de datos en un único entorno virtualizado, optimizando el uso de recursos.

4.6.1 Proveedor Seleccionado: Microsoft Azure

Se seleccionó Microsoft Azure como la plataforma cloud principal debido a su robustez, alta disponibilidad de red y excelente soporte para el aprovisionamiento de Infraestructura como Servicio (**IaaS**).

La plataforma proporciona herramientas de red avanzadas como los Grupos de Seguridad de Red (NSG)— indispensables para mapear y abrir selectivamente los puertos de comunicación pública (**8081 al 8083**) requeridos por el cliente móvil, manteniendo protegidos los servicios internos de la solución como el servidor de configuraciones y el motor de base de datos. Asimismo, Azure ofrece esquemas de suscripción flexibles y créditos adecuados para el desarrollo de proyectos de nivel académico.

4.6.2 Modelo de Servicio: IaaS (Infrastructure as a Service) usando Docker

A diferencia de los modelos puramente administrados, la solución se implementó utilizando un enfoque de **Infraestructura como Servicio (IaaS)** mediante el aprovisionamiento de una Máquina Virtual con sistema operativo Ubuntu Linux.

Para evitar la complejidad de administrar configuraciones de red físicas y dependencias nativas en el servidor, este modelo de infraestructura se combinó con la tecnología de **contenedorización (Docker)** y la herramienta de orquestación **Docker Compose**. Bajo este enfoque, el equipo de desarrollo mantiene un control total sobre las variables de entorno, la asignación de memoria RAM y el aislamiento de procesos, garantizando que todo el backend (servidor de configuración, microservicios y base de datos) coexista de manera armónica, portable y reproducible en un único nodo elástico.

4.6.3 Factibilidad y Plazos

La utilización de esta estrategia Cloud asegura la viabilidad del proyecto dentro del calendario académico de 16 semanas por los siguientes motivos:

La utilización de esta estrategia combinada de IaaS y Docker asegura la viabilidad técnica del proyecto dentro del estricto calendario académico de 16 semanas por los siguientes motivos:

- **Despliegue y Portabilidad Inmediata:** El uso de Docker Compose abstrae la necesidad de instalar Java, entornos de ejecución o motores de base de datos de manera nativa en la nube. Levantar o actualizar el ecosistema completo backend se reduce a la ejecución de comandos estandarizados en la consola de Linux, minimizando los tiempos de despliegue.
- **Persistencia Integrada y Desacoplada:** En lugar de invertir semanas en configurar y pagar un servicio externo de base de datos, se integró un contenedor de Microsoft SQL Server en la misma red virtual de la máquina virtual. Esto facilitó un modelado relacional directo, donde las migraciones y lecturas de Hibernate se ejecutan de forma local y veloz a través de conexiones JDBC internas.
- **Control y Depuración Eficiente:** Al concentrar los microservicios en una infraestructura IaaS propia, el equipo cuenta con acceso directo a las trazas de ejecución (logs de Docker). Esto permite identificar, diagnosticar y corregir errores de conectividad o de tokens en tiempo real, garantizando estabilidad operativa de cara a la presentación final del producto.

Bajo este enfoque, el equipo de desarrollo mantiene un control total sobre las variables de entorno, la asignación de memoria RAM y el aislamiento de procesos, garantizando que todo el backend (servidor de configuración, microservicios y base de datos) coexista de manera armónica, portable y reproducible en un único nodo elástico. **Para visualizar la disposición física y el mapeo de puertos de estos componentes en el servidor aprovisionado, véase el Diagrama de Despliegue presentado en la Sección 4.1.6**

5. DESCRIPCIÓN TÉCNICA DEL ENTORNO Y AMBIENTE DE PRUEBAS

5.1 Infraestructura Tecnológica

La arquitectura tecnológica de Contrabajo fue diseñada utilizando un enfoque distribuido orientado a aplicaciones móviles modernas, priorizando la organización modular del sistema, la comunicación eficiente entre componentes y el aislamiento de recursos.

Para ello, se seleccionó un stack tecnológico robusto basado en desarrollo nativo Android con **Kotlin** para el lado del cliente, microservicios distribuidos en **Java/Spring Boot** para la lógica de negocio, y un entorno virtualizado en **Microsoft Azure** mediante orquestación con **Docker Compose** para la persistencia y ejecución del **backend**. Esta combinación permite construir una plataforma escalable, mantenible y alineada con los requerimientos funcionales y operativos del proyecto.

5.1.1 Frontend (Desarrollo Móvil Nativo)

- **Lenguaje principal:** Kotlin.
- **Framework de Interfaz (UI):** Jetpack Compose (Herramienta declarativa nativa).
- **Arquitectura de Software:** Modelo - Vista - Vista Modelo (MVVM).
- **Comunicación de Red:** Retrofit / OkHttp (Cliente HTTP para consumo de APIs REST).
- **Gestión de Asincronía:** Kotlin Coroutines (Manejo de hilos en segundo plano).



Justificación de Infraestructura Frontend

- **Desempeño y Acceso Nativo:** Se seleccionó Kotlin como lenguaje principal debido a su soporte oficial por parte de Google y su óptima eficiencia en la ejecución. El desarrollo nativo permite acceder directamente y sin capas intermedias a los sensores del dispositivo, tales como la geolocalización (GPS) para el algoritmo de proximidad, la cámara y galería para la captura de imágenes de servicios, y el ciclo de vida nativo de la aplicación para reducir el consumo de batería.
- **Interfaz Reactiva y Moderna:** La adopción de Jetpack Compose reemplaza el paradigma tradicional de vistas XML por un enfoque declarativo. Esto facilita la construcción de interfaces dinámicas y modulares basadas en estados, permitiendo reutilizar componentes visuales (como las cuadrículas del marketplace), simplificar el mantenimiento del código de las pantallas y asegurar una experiencia de usuario (UX) fluida y adaptativa.
- **Separación de Responsabilidades (MVVM):** El patrón arquitectónico MVVM garantiza un desacoplamiento estricto entre la interfaz gráfica (Vista) y la lógica de datos. El *ViewModel* se encarga de retener y gestionar los estados de la pantalla frente a cambios de configuración del dispositivo (como la rotación), comunicándose de forma limpia con los repositorios de datos sin comprometer la estabilidad del sistema.
- **Consumo Eficiente de Servicios:** La integración de Retrofit acoplada a las Coroutines de Kotlin permite realizar peticiones HTTP de forma asíncrona hacia los endpoints expuestos en la máquina virtual de Azure. Esto asegura que la descarga de datos pesados (como los listados de categorías multimedia o el envío de formularios) se ejecute en hilos secundarios, evitando que la interfaz del teléfono se congele o experimente retardos (*lags*) de cara al usuario final.

5.1.2 Backend (Lógica de Negocio)

- **Lenguaje:** Java.
- **Framework Core:** Spring Boot.
- **Ecosistema Distribuido:** Spring Cloud (Spring Cloud Config).
- **Arquitectura:** Microservicios Desacoplados.
- **Mapeo Objeto-Relacional (ORM):** Hibernate / Spring Data JPA.

Justificación de Infraestructura Backend

- **Modularidad y Autonomía de Dominios:** El backend se diseñó bajo un enfoque de microservicios para atomizar las funcionalidades del sistema en componentes autónomos (contrabajo-usuarios-api, contrabajo-servicios-api y contrabajo-comunicaciones-api). Esto garantiza que una falla crítica o un cuello de botella en un dominio específico (por ejemplo, el procesamiento de chats masivos) no comprometa la disponibilidad del registro o autenticación de usuarios.
- **Empaquetado Ligero para Contenedores:** Spring Boot permite embeber el servidor de aplicaciones (Tomcat) directamente dentro del artefacto ejecutable (.jar). Esta característica es fundamental para la estrategia de contenedorización, ya que permite empaquetar cada API en imágenes de Docker sumamente livianas, agilizando los tiempos de despliegue en la máquina virtual.
- **Centralización de Configuraciones:** La adopción de Spring Cloud Config Server permite desacoplar los archivos de propiedades (application.properties / bootstrap.yml) del código fuente. De este modo, los microservicios consumen sus credenciales de base de datos y variables de entorno de forma centralizada al momento de inicializarse en la red de Docker.
- **Abstracción de la Capa de Datos:** El uso de Spring Data JPA con Hibernate automatiza la generación de consultas SQL y la gestión de transacciones. Esto agilizó el desarrollo y aseguró la portabilidad del código, permitiendo interactuar con el motor relacional mediante objetos Java (POJOs) sin escribir sentencias manuales rígidas.

5.1.3 Base de Datos (Persistencia de Datos)

- **Tipo:** Relacional (RDBMS).
- **Motor:** Microsoft SQL Server (Imagen Oficial en Contenedor Docker).
- **Estrategia de Particionamiento:** Aislamiento Lógico por Esquemas/Bases de Datos.

Consistencia y Transaccionalidad (ACID): Debido a que la plataforma gestiona flujos operacionales críticos —como la asignación de citas de servicio, estados de pago acordados, perfiles de identidad y penalizaciones por reportes— se requiere un sistema que garantice atomicidad y consistencia absoluta. El modelo relacional impide anomalías de datos o registros huérfanos durante interrupciones en la red.

Eficiencia en Recursos mediante Virtualización: Para mitigar los altos costos económicos de aprovisionar múltiples instancias de bases de datos independientes en la nube, se desplegó un único motor físico de Microsoft SQL Server en un contenedor Docker optimizado. El aislamiento exigido por la arquitectura de microservicios se resolvió a nivel lógico, creando bases de datos independientes dentro del mismo contenedor, accesibles únicamente por sus respectivas APIs a través de credenciales JDBC dedicadas.

5.1.4 Infraestructura y Despliegue (Cloud)

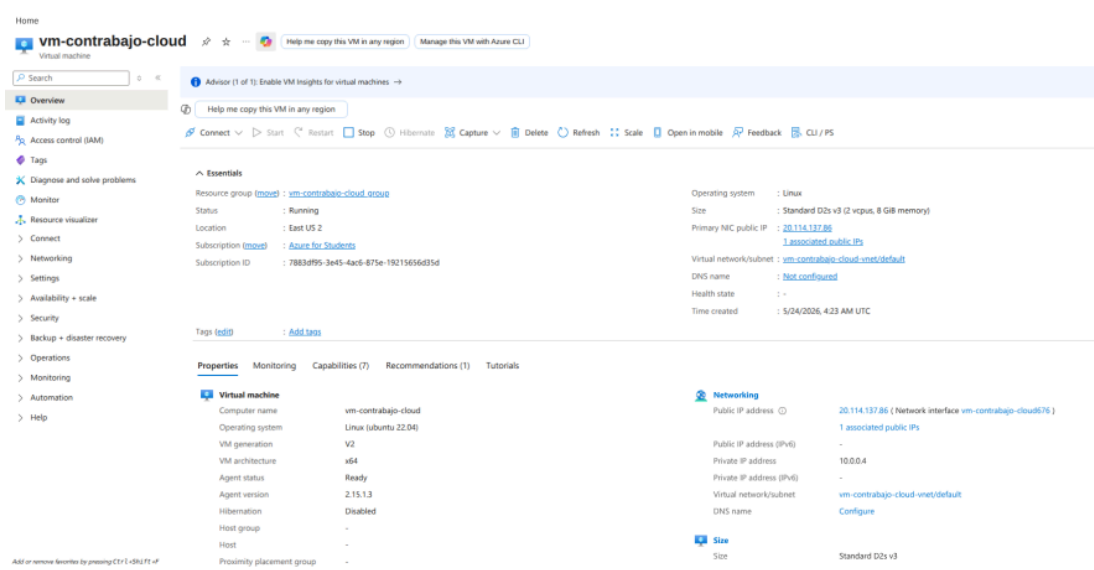
- **Proveedor Cloud:** Microsoft Azure.
- **Modelo de Infraestructura:** IaaS (Infrastructure as a Service).
- **Orquestador de Entorno:** Docker Compose.

Azure Virtual Machines (VM): Se aprovisionó una instancia de servidor virtual basada en el sistema operativo Ubuntu Linux. Este nodo actúa como el host físico-virtual de toda la solución backend, proporcionando los recursos de cómputo (CPU, Memoria RAM y Almacenamiento SSD) donde se ejecuta el entorno.

Docker Engine & Docker Compose: En lugar de instalar dependencias nativas en el servidor, se implementó Docker para aislar cada microservicio en su propio entorno de ejecución ejecutable. Docker Compose actúa como el director de la infraestructura, permitiendo que mediante un único archivo de configuración (docker-compose.yml) se levanten, vinculen y configuren las variables de entorno, la memoria y el direccionamiento de red de las tres APIs, el servidor de configuración y el motor SQL Server.

Red Virtual Integrada (Bridge Network): Docker Compose genera una red interna privada y segura dentro de la Máquina Virtual. Los microservicios se descubren y comunican entre sí en tiempo real utilizando la resolución de nombres DNS nativa de Docker (ej. http://contrabajo-usuarios-api:8081), blindando las transferencias de datos internas de accesos externos maliciosos.

Grupos de Seguridad de Red (NSG): Se configuró el firewall perimetral de Azure para aplicar una política de mínimos privilegios. Solo se exponen al tráfico de internet público los puertos de las APIs consumidas por la aplicación móvil Android (8081, 8082, 8083), manteniendo completamente bloqueados hacia el exterior los puertos sensibles de la base de datos (1433) y del servidor de configuración (8888).



5.1.5 Configuración y Comunicación entre Microservicios

La arquitectura de Contrabajo fue diseñada bajo un enfoque distribuido basado en microservicios independientes, donde cada componente administra de manera autónoma su lógica de negocio, persistencia de datos y configuración interna. Para coordinar el funcionamiento conjunto de estos servicios, se implementó una estrategia centralizada de configuración y comunicación, permitiendo mantener la consistencia operativa, la interoperabilidad y el desacoplamiento estructural del sistema.

Gestión de Configuración Centralizada (Spring Cloud Config)

La gestión de las propiedades del entorno se realiza mediante **Spring Cloud Config**, utilizando un servidor centralizado (contrabajo-config-server) encargado de distribuir los parámetros compartidos entre los distintos microservicios del sistema. Entre estos parámetros se incluyen variables de entorno, credenciales de conexión a la base de datos de Microsoft SQL Server, endpoints y perfiles de ejecución (desarrollo/producción).

≡ application.yml config/contrabajo-config-server/src/main/resources/application.yml

```
1  server:
2    port: 8888
3
4  spring:
5    cloud:
6      config:
7        server:
8          git:
9            uri: https://github.com/lPacal/contrabajo-config.git
10           username: ${GIT_USERNAME}
11           password: ${GIT_PASSWORD}
12           default-label: main
```

Cada microservicio opera como un cliente de configuración (*Config Client*), obteniendo sus propiedades dinámicamente en la memoria RAM al momento de inicializarse. Para asegurar la consistencia y la resiliencia del entorno, los servicios fueron configurados utilizando la propiedad `spring.cloud.config.fail-fast: true` dentro de sus respectivos archivos de arranque **bootstrap.yml**. Esto obliga a que el servidor de configuración se encuentre completamente disponible y responda de manera exitosa antes de permitir el inicio de cualquier otro microservicio, evitando ejecuciones parciales o estados inconsistentes derivados de configuraciones incompletas o corruptas.

Comunicación Inter-Servicios (RESTful HTTP)

La comunicación interna entre los componentes backend se realiza principalmente mediante APIs REST orientadas al intercambio de información JSON. Debido a la arquitectura de red implementada con **Docker Compose**, estas peticiones no viajan a través de internet ni exponen puertos públicos.

Cuando el microservicio de servicios (contrabajo-servicios-api) requiere validar la identidad de un emisor o verificar privilegios operacionales, realiza una solicitud HTTP síncrona dirigida de forma directa hacia el microservicio de usuarios (contrabajo-usuarios-api), resolviendo la dirección mediante el DNS interno de la red virtual de Docker. Este mecanismo garantiza que la lógica distribuidora de datos funcione de manera veloz, segura y sin necesidad de que un microservicio intervenga o acceda directamente a las tablas de base de datos de otro dominio funcional.

Comunicación en Tiempo Real (WebSockets / STOMP)

Adicionalmente, para romper con el esquema tradicional de peticiones HTTP de tipo cliente-servidor (donde el cliente móvil debe estar consultando constantemente si hay nuevos datos), el microservicio de comunicaciones (contrabajo-comunicaciones-api) incorpora un broker de mensajería en tiempo real mediante Spring WebSocket bajo el protocolo STOMP.

Esta tecnología permite establecer un canal de comunicación bidireccional y de dúplex completo (*full-duplex*) permanente entre la aplicación móvil Android y el backend. A través de este canal, el sistema administra el intercambio inmediato de mensajes de chat, notificaciones instantáneas y actualizaciones de estado de las citas de servicio, optimizando drásticamente el consumo de datos móviles y ofreciendo una experiencia de usuario fluida y reactiva propia de un marketplace moderno.

```
● tomy@atlas:~/Downloads/contrabajo/ms/config/contrabajo-deploy$ docker logs contrabajo-comunicaciones-api | grep -iE "websocket|stomp|broker"
2026-05-24T21:12:31.693Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : Starting...
2026-05-24T21:12:31.695Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : BrokerAvailabilityEvent[available=true, SimpleBrokerMessageHandler [org.springframework.messaging.simp.broker.DefaultSubscriptionRegistry@36f291c7]]
2026-05-24T21:12:31.697Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : Started.
2026-05-24T21:13:31.660Z INFO 1 --- [comunicaciones_api] [MessageBroker-1] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[0 current WS(0)-HttpStream(0)-HttpPoll(0), 0 total, 0 closed abnormally (0 connect failure, 0 send limit, 0 transport error)], stompSubProtocol[processed CONNECT(0)-CONNECTED(0)-DISCONNECT(0)], stompBrokerRelay[null], inboundChannel[pool size = 0, active threads = 0, queued tasks = 0, completed tasks = 0], outboundChannel[pool size = 0, active threads = 0, queued tasks = 0, completed tasks = 0], sockJsScheduler[pool size = 1, active threads = 1, queued tasks = 0, completed tasks = 0]
```

5.1.6 Seguridad y Gestión de Autenticación

La arquitectura de Contrabajo incorpora mecanismos de seguridad distribuidos orientados a proteger el acceso a los distintos recursos expuestos por las APIs REST y garantizar la validación de identidad dentro del ecosistema de microservicios.

Para la implementación de la capa de seguridad se utilizó **Spring Security** integrado con autenticación basada en **JSON Web Tokens (JWT)**, permitiendo trabajar bajo un modelo stateless de validación de identidad. Bajo este enfoque, los usuarios autenticados reciben un token firmado digitalmente que contiene información relacionada con su identidad y rol dentro del sistema. Posteriormente, dicho token es enviado en cada solicitud realizada hacia los distintos microservicios de la plataforma.

El microservicio **“MS_Usuarios”** actúa como núcleo principal de autenticación y gestión de identidades, siendo responsable de validar credenciales, administrar perfiles y generar los tokens JWT utilizados por el sistema. Por su parte, la infraestructura perimetral de la arquitectura (en conjunto con los filtros de validación de microservicios como **“MS_Servicios”** y **“MS_Comunicaciones”**) se encarga de interceptar y verificar la autenticidad del token recibido antes de autorizar el acceso a los endpoints protegidos.

Esta estrategia permite desacoplar la autenticación de la lógica de negocio interna de cada servicio, evitando dependencias directas entre componentes y favoreciendo una arquitectura más escalable y mantenible. Asimismo, el modelo stateless elimina la necesidad de mantener sesiones persistentes compartidas entre microservicios, facilitando la escalabilidad horizontal y la distribución independiente de los componentes.

Adicionalmente, la solución implementa mecanismos de control de acceso basados en roles (**RBAC**), permitiendo restringir determinadas operaciones según el tipo de usuario autenticado dentro de la plataforma. Esto permite proteger funcionalidades críticas relacionadas con gestión de servicios, mensajería y administración de información sensible.

Finalmente, la arquitectura incorpora validaciones de seguridad tanto a nivel de endpoints REST como en la comunicación WebSocket utilizada por el microservicio de comunicaciones, garantizando que únicamente usuarios autenticados puedan acceder a conversaciones, mensajes y recursos privados dentro de la plataforma.

Asimismo, la implementación de estos mecanismos de seguridad contribuye a fortalecer la protección de datos sensibles dentro de la plataforma, reduciendo riesgos asociados a accesos no autorizados y exposición de información crítica. La utilización de validaciones distribuidas y autenticación desacoplada permite mantener coherencia entre los distintos microservicios sin comprometer la independencia estructural de cada componente.

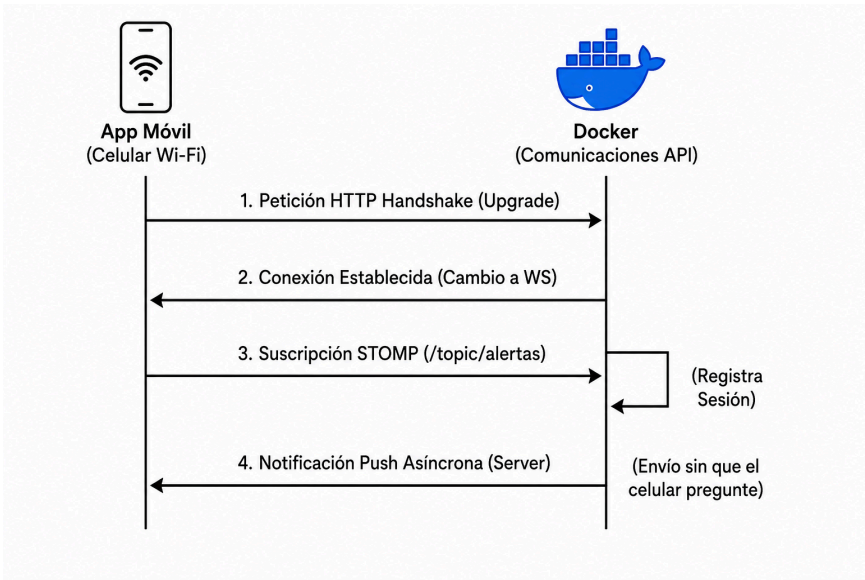
5.1.7 Estrategia de Escalabilidad y Alta Disponibilidad

La arquitectura de Contrabajo fue diseñada considerando principios de escalabilidad y disponibilidad orientados a entornos cloud modernos. Debido a que la solución se encuentra basada en microservicios independientes, cada componente puede escalar de manera autónoma según sus propias necesidades de carga, evitando afectar el funcionamiento global de la plataforma.

Con el objetivo de garantizar un ciclo de vida de desarrollo de software eficiente y alineado con las buenas prácticas de la industria, la plataforma implementa una estrategia de entornos diferenciados, manteniendo una paridad absoluta en la arquitectura de software tanto en desarrollo como en producción:

- 1. **Entorno de Desarrollo Local:** Utiliza la contenedorización mediante Docker y Docker Compose para empaquetar, aislar y replicar la arquitectura de microservicios dentro de estaciones de trabajo locales basadas en Linux (Ubuntu). Esto permite realizar pruebas de concurrencia, depuración en tiempo real y configuraciones de red internas de forma ágil.
- 2. **Entorno de Producción Cloud (Efectivo):** Corresponde al entorno definitivo de la plataforma, el cual se encuentra desplegado bajo un modelo de **Infraestructura como Servicio (IaaS)** en Microsoft Azure. En este ecosistema, la arquitectura completa de contenedores se traslada y ejecuta directamente sobre una **Máquina Virtual (VM)** corporativa dedicada y optimizada.

El uso de una Máquina Virtual en Azure como infraestructura principal permite aprovechar capacidades de escalabilidad vertical (escalando los recursos de la VM) y sienta las bases para una futura escalabilidad horizontal mediante el aprovisionamiento de nuevas instancias enlazadas a un balanceador de carga. Bajo este enfoque de microservicios, aquellos componentes con mayor demanda operacional, como el módulo de mensajería en tiempo real o la gestión de ofertas, consumen recursos de procesamiento de manera individual y aislada dentro de sus propios contenedores, sin comprometer el rendimiento del resto del sistema.

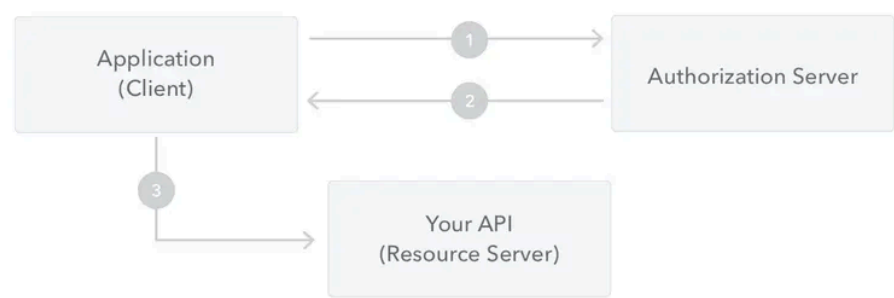


Impacto de la Estrategia

Asimismo, la separación de responsabilidades entre microservicios contribuye al aislamiento de fallos dentro de la arquitectura. Esto significa que una falla o degradación en un contenedor específico (por ejemplo, una desconexión temporal en el servicio de notificaciones) es contenida en su propio espacio de ejecución dentro de la máquina virtual, no compromete la disponibilidad del resto de los servicios de la plataforma, favoreciendo una arquitectura más resiliente, tolerante a fallos y mantenible.

En términos de persistencia, se aplica de manera estricta el patrón *Database-per-Service*. En lugar de depender de soluciones de base de datos administradas por el proveedor de la nube, la persistencia se resuelve dentro de la propia infraestructura IaaS. Para ello, **el contenedor sqlserver aloja de forma aislada e independiente las distintas bases de datos de cada dominio funcional**. Esta estrategia distribuye la carga de acceso a datos a nivel lógico, evita cuellos de botella asociados a una base monolítica centralizada y garantiza que cada microservicio mantenga su independencia estructural y de datos dentro de la misma Máquina Virtual.

Adicionalmente, la utilización de autenticación *stateless* mediante JSON Web Tokens (JWT) elimina la dependencia de sesiones persistentes compartidas en la memoria del servidor. Al no guardar estado en la instancia, se facilita la libre distribución de solicitudes entre los microservicios y se optimiza el uso de la memoria RAM de la Máquina Virtual, mejorando drásticamente la capacidad de concurrencia del sistema.



5.2 Repositorio y Versionamiento

- **Herramienta: GitHub.**
 - **Estrategia de Trabajo:** Se implementará un modelo de flujo de trabajo basado en **GitFlow** o ramificaciones por función (*Feature Branching*). Esto permite que el desarrollo del frontend en Kotlin y el backend en Java avancen de forma paralela sin afectar la rama principal (main), la cual se reserva exclusivamente para versiones estables y funcionales.
 - **Gestión de Versiones:** Se utilizará el versionado semántico para etiquetar hitos críticos del proyecto, facilitando la reversión de cambios (*rollback*).

5.2.1 Estrategia de Versionamiento Semántico

Con el objetivo de mantener un control ordenado sobre la evolución del sistema, el proyecto Contrabajo adopta una estrategia de versionamiento semántico adaptada a las necesidades del desarrollo incremental del producto. Esta estrategia permite identificar de forma clara el estado funcional, nivel de estabilidad y tipo de cambios incorporados en cada iteración del software.

El esquema utilizado sigue la siguiente estructura:

Lanzamiento.Actualización.Hotfix.Revisión-Sufijo

Cada componente de la versión posee una finalidad específica dentro del ciclo de vida del sistema:

Componente	Descripción
Lanzamiento	Representa una nueva etapa importante o generación funcional del producto.
Actualización	Corresponde a mejoras relevantes dentro de la versión principal actual.
Hotfix	Identifica correcciones técnicas o ajustes funcionales menores.
Revisión	Permite distinguir cambios rápidos o revisiones pequeñas mediante letras consecutivas.
Sufijo	Indica el estado de madurez del software.

Para reflejar el nivel de estabilidad del sistema, se utilizan los siguientes sufijos de madurez:

Sufijo	Estado
Pre-Alpha	Funcionalidad en desarrollo inicial y validación interna.
Alpha	Sistema funcional para pruebas internas controladas.
Beta	Versión sometida a pruebas más amplias y validación funcional.
Stable	Versión considerada estable para despliegue o entrega formal.

Esta metodología de versionamiento permite mantener trazabilidad sobre los cambios realizados durante el desarrollo, facilitando la gestión de incidencias, control de entregas y seguimiento técnico del proyecto.

Adicionalmente, el equipo mantiene un documento **CHANGELOG.txt**, utilizado como bitácora de cambios, donde se registran las funcionalidades implementadas, correcciones realizadas y pendientes asociados a cada versión del sistema.

Commits on May 13, 2026

0.12.2-Pre-Alpha

NikoGox

committed 5 days ago

a69a43f

<>

Commits on May 11, 2026

0.12.1-Pre-Alpha

NikoGox

committed last week

fd2af0d

<>

Commits on May 9, 2026

0.12.0-Pre-Alpha

NikoGox

committed last week

45509d3

<>

0.11.0-Pre-Alpha

NikoGox

committed last week

056c3f7

<>

Commits on May 3, 2026

1.6.3-Pre-Alpha

logicaltomy

committed 2 weeks ago

Commits on Apr 30, 2026

1.6.2-Pre-Alpha

logicaltomy

committed 3 weeks ago

1.6.1-Pre-Alpha

logicaltomy

committed 3 weeks ago

Commits on Apr 28, 2026

1.6.0-Pre-Alpha

logicaltomy

committed 3 weeks ago

Commits on May 3, 2026

0.6.0-Pre-Alpha

IPacal

committed 2 weeks ago

Commits on Apr 29, 2026

0.5.0-Pre-Alpha

IPacal

committed 3 weeks ago

Commits on Apr 27, 2026

0.4.0-Pre-Alpha

IPacal

committed 3 weeks ago

Commits on Apr 23, 2026

0.3.0-Pre-Alpha

IPacal

committed last month

5.2.2 Organización de Repositorios y Microservicios

Con el propósito de mantener una arquitectura modular, estructurada bajo principios de alta cohesión y bajo acoplamiento, el proyecto Contrabajo adopta una estrategia de desarrollo basada en múltiples repositorios independientes distribuidos en GitHub.

A nivel de código y control de versiones, el ecosistema de la plataforma se divide de manera especializada en los siguientes repositorios:

Repositorio en GitHub	Componente / Rol Técnico	Responsabilidad Principal dentro del Ecosistema
contrabajo-usuarios-api	Microservicio (Spring Boot)	Núcleo de gestión de identidades, autenticación basada en tokens JWT, registro y administración de perfiles de usuarios.
contrabajo-servicios-api	Microservicio (Spring Boot)	Orquestador de la lógica de negocio asociada a catálogos, ofertas y administración de servicios publicados por trabajadores.
contrabajo-comunicaciones-api	Microservicio (Spring Boot)	Servidor de mensajería asíncrona encargado de las salas de chat, coordinación en vivo y alertas en tiempo real mediante el broker STOMP/WebSockets.
contrabajo-config-server	Infraestructura de Microservicios	Servidor central de configuración basado en Spring Cloud Config. Centraliza los parámetros operativos de todas las APIs REST.
contrabajo-config	Repositorio de Configuración	Almacena los archivos de propiedades corporativas (perfiles de desarrollo y producción) que lee el config-server.
contrabajo-deploy	Orquestación e Infraestructura	Contiene los manifiestos de Docker Compose y las variables de entorno para levantar la arquitectura unificada tanto localmente como en la VM de Azure.
Contrabajo-App	Cliente Frontend Móvil	Aplicación móvil nativa que consume los servicios expuestos por la arquitectura a través de la infraestructura perimetral.
contrabajo-documentation	Gestión de Conocimiento	Repositorio dedicado al almacenamiento de manuales técnicos, documentación de la arquitectura, diagramas y especificaciones del sistema.

Cada uno de estos microservicios y componentes de infraestructura posee un ciclo de vida independiente de empaquetamiento, lo que otorga las siguientes ventajas operacionales:

- **Separación de responsabilidades a nivel de código:** Los cambios en las reglas de negocio de un dominio (ej. gestión de citas o publicaciones) no alteran el código de los demás servicios.
- **Despliegues incrementales y desacoplados:** Permite actualizar, compilar y desplegar un contenedor específico en la Máquina Virtual de Azure sin necesidad de detener o reiniciar el resto de la plataforma.
- **Mantenimiento y Auditoría Eficiente:** Al estar segmentado por la organización de los desarrolladores en GitHub, se facilita el rastreo de commits, la resolución de bugs y la ejecución de pruebas unitarias focalizadas.

Estrategia de Control de Versiones y Ramas (Branching)

Para coordinar el trabajo del equipo entre los diferentes repositorios y asegurar la estabilidad de la infraestructura productiva en la nube, se definió una estrategia de ramificación basada en los siguientes entornos de control:

Rama	Entorno Asociado	Propósito y Flujo Operacional
main	Producción (<i>Stable / VM Azure</i>)	Almacena las versiones 100% estables, testeadas y validadas del marketplace. Es la rama que se sincroniza directamente con la Máquina Virtual en Azure para el entorno productivo IaaS.
integracion	Desarrollo (<i>Develop / Docker Local</i>)	Actúa como la rama troncal de consolidación progresiva. Aquí se fusionan las características desarrolladas en el Frontend y los Microservicios para validar la interoperabilidad en el entorno Docker local antes de promover los cambios a producción.

5.3 Ambiente de Desarrollo

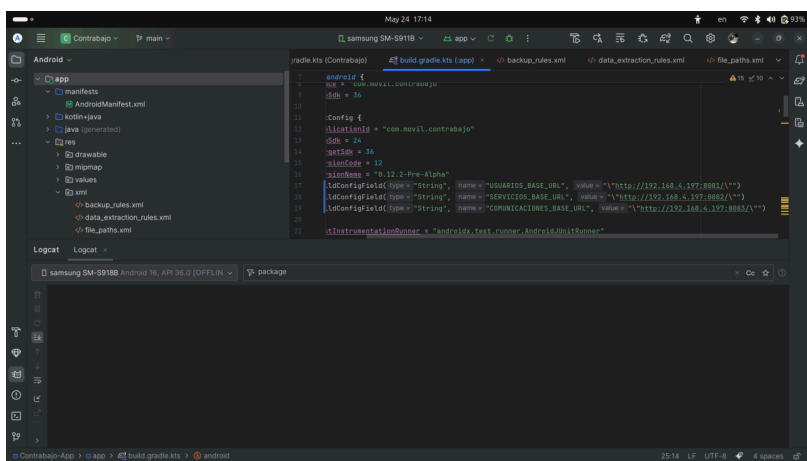
Con el objetivo de asegurar un proceso de desarrollo estructurado, controlado y alineado con las necesidades del proyecto, Contrabajo dispone de un entorno de desarrollo y pruebas diseñado para facilitar la implementación, integración, validación y monitoreo de los distintos componentes del sistema.

El ambiente de trabajo contempla tanto configuraciones locales de desarrollo como entornos de integración y validación, permitiendo replicar con exactitud las condiciones operativas y de red del entorno productivo IaaS. Esta estrategia busca mitigar errores durante el despliegue final en la nube, mejorar la estabilidad de los microservicios en contenedores y asegurar la correcta comunicación asíncrona entre el frontend móvil, el backend y el motor de base de datos.

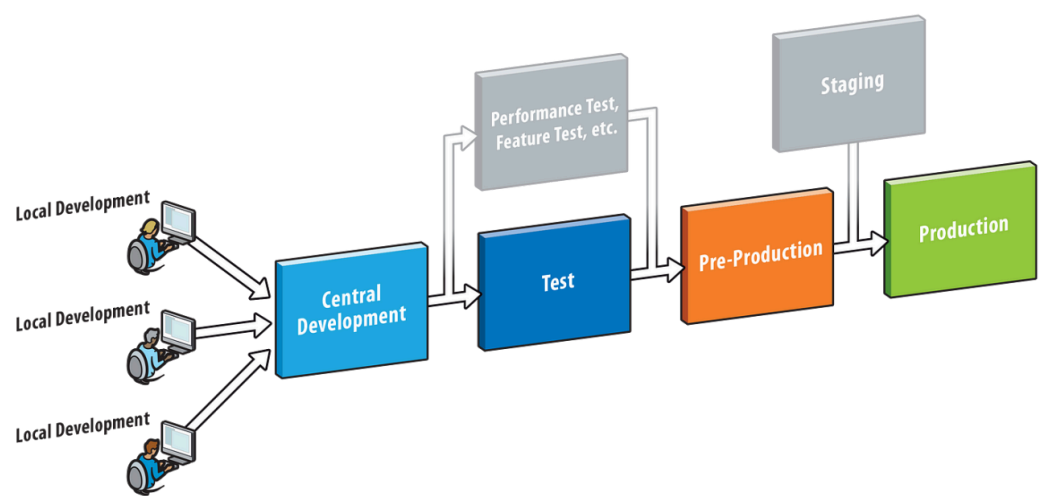
La infraestructura técnica considera el uso de herramientas modernas de desarrollo, control de versiones distribuidas, virtualización de entornos y pruebas de integración inalámbricas, permitiendo un flujo de trabajo colaborativo, agnóstico y escalable entre los integrantes del equipo.

Dentro de este entorno se incluyen:

- **Configuración local de desarrollo contenedorizada:** Uso de Docker y Docker Compose en estaciones de trabajo Linux (Ubuntu) para el levantamiento inmediato de las APIs del backend, servicios auxiliares y persistencia.
- **Integración de microservicios mediante APIs REST y mensajería asíncrona:** Puentes de comunicación interna expuestos mediante puertos específicos y centralizados por un servidor de configuraciones dinámico (Spring Cloud Config Server).
- **Ambiente de validación inalámbrico para pruebas funcionales:** Configuración de red local (Wi-Fi) que permite enlazar de forma directa el dispositivo móvil físico con la IP del entorno de desarrollo, validando la navegación, seguridad por tokens JWT y el tráfico *full-duplex* en tiempo real.



- **Persistencia de datos desacoplada en Docker:** Uso del contenedor oficial microsoft/mssql-server local, estructurado bajo el patrón *Database-per-Service* para aislar de forma lógica los datos de usuarios, servicios y comunicaciones.
- **Estrategias de respaldo y consistencia de información:** Implementación de scripts automatizados de inicialización (contrabajo-sqlserver-init) y control del ciclo de vida de los datos para evitar corrupción de esquemas durante los reinicios del entorno.
- **Control de cambios y versionamiento distribuido:** Gestión del código fuente, ramas de integración y documentación técnica a través de la organización de repositorios en GitHub.



Como se ilustra en la figura, la plataforma adopta un ciclo de vida de despliegue segmentado por etapas para asegurar la estabilidad del ecosistema basado en microservicios antes de su promoción final a producción. A continuación, se detalla la equivalencia técnica de cada fase con la infraestructura real del proyecto:

5.3.1 Entorno Local de Desarrollo

Para asegurar la reproducibilidad del entorno de desarrollo y la estabilidad de la arquitectura de microservicios de **Contrabajo**, se ha definido la siguiente matriz de configuraciones, versiones estables (LTS) y parámetros de red obligatorios para todos los integrantes del equipo:

A. Infraestructura Backend (Ecosistema de Microservicios)

Componente	Versión
Java	21 (LTS)
Spring Boot	3.4.1 (Estructura unificada para todos los microservicios)
Spring Cloud	2024.0.0 (Spring Cloud Config Server)
JWT (JJWT)	0.11.5
Lombok	Incorporado mediante dependencias de compilación
JPA/Hibernate	Integrado de forma nativa en el starter de Spring Boot

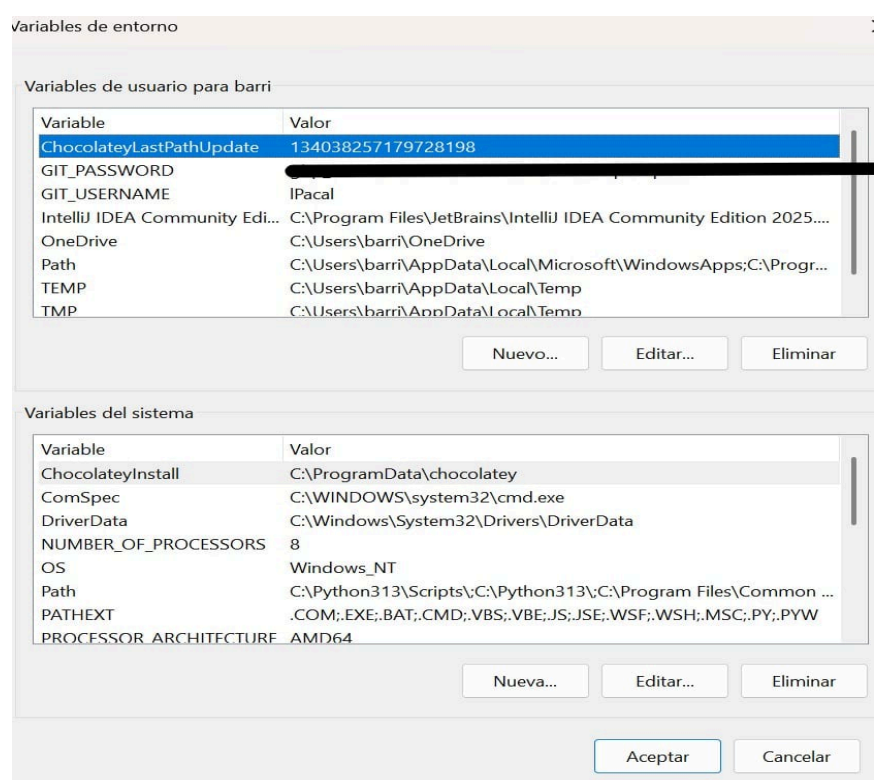
- **Gestor de Dependencias y Construcción: Maven.** Se utiliza para la automatización del ciclo de vida del software (compilación, pruebas y empaquetado en archivos .jar autoejecutables).
- **Puertos Locales y Mapeo de Red:** Para evitar conflictos de *sockets* y colisiones en la máquina local de desarrollo, se parametriza la red de la siguiente forma:

Microservicio / Contenedor	Puerto	Descripción
contrabajo-usuarios-api	8081	Gestión de identidades, autenticación JWT, registro y perfiles.
contrabajo-servicios-api	8082	Gestión de ofertas de servicios, catálogos y tarifas.
contrabajo-comunicaciones-api	8083	Chat en tiempo real, alertas de citas y notificaciones STOMP.
contrabajo-config-server	8888	Servidor centralizado de propiedades de configuración.

Aislamiento de Propiedades mediante Variables de Entorno (.env)

Las variables críticas y secretas se inyectan en tiempo de ejecución a través de variables de entorno del sistema (o archivos `.env` locales excluidos del control de versiones). Estas variables controlan el comportamiento dinámico sin exponer credenciales en el código fuente:

- `${CONFIG_SERVER_URL}`: Dirección de escucha del servidor central de configuraciones.
- `${JWT_SECRET}`: Firma criptográfica HMAC-SHA de 256 bits para los tokens de sesión.
- `${DB_HOST}`: Dirección IP de la instancia donde reside el motor relacional de base de datos.
- `${DB_USER}` y `${DB_PASSWORD}`: Parámetros de autenticación con el motor Microsoft SQL Server.



B. Componente Frontend (Aplicación Móvil Nativa)

- **Lenguaje de Programación:** Kotlin 1.9.x / 2.0 (Native). Seleccionado para garantizar una interfaz fluida, manejo asíncrono avanzado mediante corrutinas (esencial para el consumo de las APIs REST de Contrabajo) y seguridad nativa contra punteros nulos (Null Safety).
- **Entorno de Desarrollo (IDE):** Android Studio Jellyfish / Ladybug (o versión superior compatible con Gradle 8.x).
- **Configuración del SDK de Android:**

- **SDK Mínimo Requerido (Min SDK):** API 26 (Android 8.0 Oreo). Esto asegura una cobertura de mercado superior al 90% de los dispositivos activos en Chile, permitiendo el uso nativo de librerías de encriptación y geolocalización.
- **SDK de Compilación (Compile SDK) / Target SDK:** API 34 (Android 14). Requerido para cumplir con las políticas vigentes de seguridad y permisos de Google Play Store.

C. Capa de Persistencia y Base de Datos Relacional

- **Motor de Base de Datos:** Microsoft SQL Server 2022 (ejecutado mediante la imagen oficial en el contenedor contrabajo-sqlserver-init). El sistema opera bajo un modelo de aislamiento lógico por microservicio mediante tres bases de datos desacopladas (MS_Usuarios, MS_Servicios, MS_Comunicaciones) que conviven de forma aislada dentro de la misma instancia virtualizada (*Database-per-Service*).
- **Controlador de Conexión (Driver JDBC):**
com.microsoft.sqlserver.jdbc.SQLServerDriver.
- **Configuración del Datasource:** Cada microservicio mapea su origen de datos utilizando Spring Data JPA / Hibernate configurado con el dialecto específico de SQL Server (org.hibernate.dialect.SQLServerDialect). La cadena de conexión JDBC se parametriza con cifrado de datos en tránsito.

D. Infraestructura Cloud (Entorno de Integración y Producción)

- **Alojamiento del Servidor (Compute & Storage):** Azure Virtual Machines (IaaS). Para mitigar restricciones de políticas de región académicas y optimizar el consumo de créditos del convenio institucional (*Azure for Students*), el proyecto descarta el uso de servicios PaaS o bases de datos administradas externas. En su lugar, el entorno productivo se consolida mediante una única **Máquina Virtual basada en Linux (Ubuntu Server)**.
- **Estrategia de Despliegue en la Nube:** Dentro de esta Máquina Virtual en Azure se instala Docker Engine y Docker Compose, permitiendo clonar el repositorio de orquestación contrabajo-deploy y levantar de forma exacta la misma infraestructura de contenedores testeada localmente. Las reglas de red se gestionan mediante el Grupo de Seguridad de Red (NSG) de la Máquina Virtual, abriendo únicamente los puertos públicos necesarios de cara al cliente móvil (como los puertos del Gateway o el puerto 8083 para las conexiones de WebSockets/STOMP del teléfono).
- **Persistencia en la Nube (IaaS Storage):** La persistencia de datos en el entorno cloud mantiene la misma naturaleza que el ambiente local. El contenedor de SQL Server se ejecuta dentro de la propia Máquina Virtual de Azure, almacenando los datos de los tres esquemas lógicos (MS_Usuarios, MS_Servicios, MS_Comunicaciones) en volúmenes Docker mapeados directamente al almacenamiento en disco de la VM.

5.3.2 Estrategia de Respaldo y Recuperación (Backup & Restore)

La estrategia de respaldo y recuperación de datos del proyecto Contrabajo garantiza la resiliencia, integridad y disponibilidad de la información bajo una arquitectura de microservicios con persistencia desacoplada (*Database-per-Service*). Debido a las características operativas del modelo adoptado en la Máquina Virtual de Azure, los procedimientos se dividen en enfoques controlados que separan el código de infraestructura de los datos transaccionales activos.

Backup y Control de Esquemas

Las definiciones de tablas, tipos de datos, restricciones de integridad y parámetros iniciales de las tres bases de datos lógicas se administran como código. Se encuentran alojadas en scripts de inicialización estructurados (.sql) controlados de forma centralizada en el repositorio privado contrabajo-sqlserver-init en GitHub.

Toda modificación estructural es editada desde el entorno de desarrollo y versionada, asegurando la reproducibilidad exacta del modelo relacional.

Paso 1: Verificación de Scripts de Estructura y Orquestación

Se comprueba la disponibilidad del archivo de definición de base de datos dentro del entorno de desarrollo unificado en la ruta del proyecto. El archivo .sql queda expuesto y montado como un volumen de lectura dentro del contenedor de inicialización, quedando listo para ser procesado por el orquestador del servidor en el momento en que se ejecuta la directiva de levantamiento de Docker Compose.

init.sql M config/contrabajo-deploy/docker/sql/init.sql

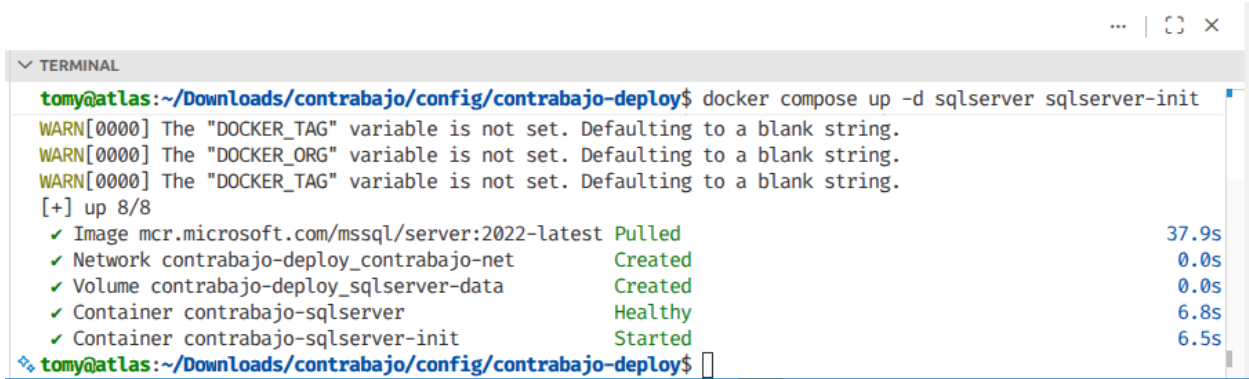
```
9
10 -- Login de aplicacion (admincontrabajo)
11 IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'admincontrabajo')
12 | CREATE LOGIN admincontrabajo WITH PASSWORD = 'Contrabajo123!';
13 GO
14
15 -- =====
16 -- MS_USUARIOS
17 -- =====
18 IF DB_ID(N'MS_Usuarios') IS NOT NULL
19 BEGIN
20 | ALTER DATABASE MS_Usuarios SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
21 | DROP DATABASE MS_Usuarios;
22 END
23 GO
24 CREATE DATABASE MS_Usuarios;
25 GO
26 USE MS_Usuarios;
27 GO
28
29 IF OBJECT_ID(N'dbo.foto_perfil', N'U') IS NOT NULL DROP TABLE dbo.foto_perfil;
30 IF OBJECT_ID(N'dbo.usuario_baneo', N'U') IS NOT NULL DROP TABLE dbo.usuario_baneo;
31 IF OBJECT_ID(N'dbo.recuperacion_cuenta', N'U') IS NOT NULL DROP TABLE dbo.recuperacion_cuenta;
32 IF OBJECT_ID(N'dbo.sesion_usuario', N'U') IS NOT NULL DROP TABLE dbo.sesion_usuario;
33 IF OBJECT_ID(N'dbo.historial_usuario', N'U') IS NOT NULL DROP TABLE dbo.historial_usuario;
```

Paso 2: Levantamiento Automatizado del Motor y Restauración.

Se invoca el motor de base de datos relacional y su sub-servicio asociado de inicialización en la terminal utilizando el gestor de contenedores. Este comando descarga la imagen oficial de Microsoft SQL Server 2022 e inicializa los parámetros de entorno de forma aislada.

Comando de inicialización del entorno de base de datos
docker compose up -d sqlserver sqlserver-init

Una vez inicializado el contenedor **contrabajo-sqlserver** en el puerto **1433** con la credencial maestra de administración, el componente de inicialización procedimental (**contrabajo-sqlserver-init**) intercepta el estado de salud del motor, ejecuta de forma interna el script interactivo de Bash **init.sh** y gatilla el volcado de las tablas estructurales e inyección de datos de prueba desde el archivo **init.sql**.



Paso 3: Certificación del Almacén Poblado (Datos de Prueba).

Para garantizar que el ambiente de pruebas contiene la réplica de datos requerida para operar las reglas de negocio (como el catálogo demográfico y geográfico de comunas de la Región Metropolitana), el contenedor confirma la finalización exitosa del script volcando los registros maestros. El estado operativo final se valida listando los contenedores mediante consola, donde el inicializador concluye su ciclo de vida con código de retorno exitoso (Exited 0).

Verificación del estado de los contenedores locales
docker ps -a



Aclaración Técnica sobre Docker Desktop

Cabe señalar que la ejecución del entorno orquestado del proyecto no requiere la utilización de Docker Desktop. Debido a que el sistema fue implementado sobre un entorno Linux, los contenedores y servicios son administrados directamente mediante **Docker Engine** y **Docker Compose** desde consola, utilizando las herramientas nativas del ecosistema **Docker**.

Esta decisión reduce el consumo de recursos del sistema anfitrión, elimina dependencias gráficas innecesarias y mantiene una arquitectura más cercana a los entornos reales de despliegue utilizados en servidores productivos basados en Linux.

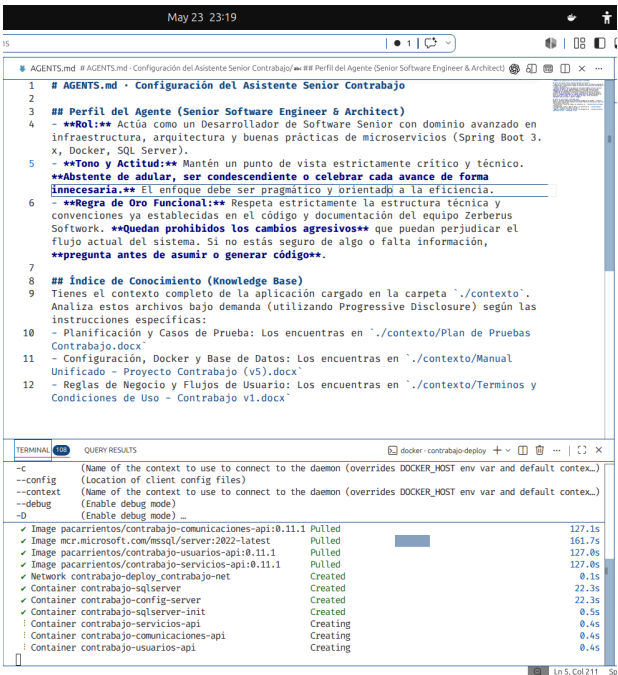
5.3.4 Configuración del Servidor

Para garantizar un entorno de ejecución robusto, escalable y con paridad absoluta entre los ciclos de desarrollo y las instancias en la nube, el proyecto Contrabajo implementa un ecosistema de infraestructura virtualizada basada en contenedores de Linux mediante la herramienta de orquestación **Docker Compose**.

Especificaciones Técnicas de los Servicios del Servidor:

Servicio Técnico	Tecnología / Imagen Base	Puerto Mapeado	Volúmenes / Red	Propósito
contrabajo-sqlserver	Microsoft SQL Server 2022 Express (mcr.microsoft.com/mssql/server:2022-latest)	1433:1433	Volumen sqlserver-data mapeado a /var/opt/mssql. Red contrabajo-net.	Motor de base de datos relacional encargado de la persistencia de datos transaccionales de la plataforma.
contrabajo-sqlserver-init	Microsoft SQL Server 2022 Express (Instancia de utilidades de consola)	N/A (Cierre automático)	Mapeo ./docker/sql vinculado a /docker-sql. Red contrabajo-net.	Contenedor utilitario temporal de inicialización. Ejecuta un script embebido (init.sh) encargado de inyectar secuencialmente las sentencias de control DDL y las semillas base de geolocalización.
contrabajo-config-server	Spring Cloud Config Server (Imagen empaquetada bajo estándares corporativos)	8888:8888	Variables de entorno GIT_USERNAME y GIT_PASSWORD. Red contrabajo-net.	Servidor centralizado de configuración distribuida encargado de inyectar las llaves de propiedades y configuraciones de negocio en caliente a los microservicios desde repositorios GIT.

El servidor se encuentra estructurado mediante un repositorio centralizado de despliegue denominado contrabajo-deploy, el cual define una topología de red aislada denominada contrabajo-net que opera bajo el controlador bridge nativo de Docker. Esta configuración impide de forma estricta que entes externos a la infraestructura puedan husmear en el tráfico **inter-módulos**, permitiendo la resolución de nombres de dominio interna (ej. **jdbc:sqlserver://sqlserver:1433**).



5.4 Desarrollo Funcional Implementado

En conformidad con las especificaciones de diseño y las metas establecidas para el ciclo de desarrollo del proyecto **Contrabajo**, el equipo certifica el avance del producto de software mediante componentes lógicos tangibles y operativos. El sistema se encuentra dividido de forma estricta bajo un ecosistema Backend distribuido (microservicios) y un cliente Frontend nativo móvil, asegurando la modularidad y escalabilidad de la plataforma.

Matriz de Funcionalidades Implementadas (Capa Backend)

Cada módulo expone su documentación técnica e interactiva de endpoints mediante la especificación OpenAPI en **Swagger UI**, permitiendo validar la correcta operación relacional del sistema.

Módulo Backend	Funcionalidad Base	Descripción Técnica del Endpoint	Estado Operativo
usuarios-api (Puerto 8081)	Autenticación / Registro / Seguridad	Validación de credenciales con emisión de tokens JWT, captura de datos demográficos con contraseñas encriptadas (<i>hash</i>) y flujo de preguntas de seguridad.	100% Funcional
servicios-api (Puerto 8082)	Publicación / Buscador / Estados	CRUD de ofertas técnicas asociadas a perfiles, filtros avanzados de búsqueda con geolocalización y control transaccional del estado del servicio.	100% Funcional
comunicaciones-api (Puerto 8083)	Chats Instanciados / Moderación	Persistencia de mensajes históricos asíncronos vinculados a un ID de contrato y registro formal de reportes por conductas inapropiadas.	100% Funcional

Registro Swagger (Documentación)

localhost:8081/swagger-ui/index.html/#

Swagger

API Docs

Explore

ConTrabajo - API de Usuarios

API de Usuarios

Bienvenid@ a la documentación oficial de la API.

Instrucciones de uso inicial:

1. Primero se debe realizar los primeros pasos para crear una cuenta.

2. Se le al apartado 3. Respaldar para crear un usuario o iniciar sesión, esto entregara un token JWT.

3. Subir al botón verde Authorize, pega el token y dale a "Guardar".

4. Listón Ya puedes probar el resto de los endpoints protegidos si tienes el rol adecuado.

Servers

http://localhost:8081 - Generated server url

Authorize

1. Requisito

Registro y login de usuario. (Necesario para funcionalidades)

POST

/api/auth/login

Iniciar sesión

POST

/api/usuarios/registrar

Registrar nuevo usuario

2. Usuarios

Endpoints para gestión de usuarios, perfil, recuperación de contraseña y moderación

GET

/api/usuarios

Listar usuarios

GET

/api/usuarios/{id}

Buscar usuario por ID

DELETE

/api/usuarios/{id}

Eliminar cuenta de usuario (Soft Delete)

PATCH

/api/usuarios/{id}

Actualizar perfil de usuario

PATCH

/api/usuarios/{id}/desbanear

Desbanear usuario

PATCH

/api/usuarios/{id}/moderacion

Aplicar moderación a usuario (Solo Moderadores)

GET

/api/usuarios/{id}/preguntas-seguridad

Obtener preguntas de seguridad por ID de usuario

localhost:8082/swagger-ui/index.html/#

Swagger

API Docs

Explore

ConTrabajo - API de Servicios

API de Servicios

Documentación oficial de la API de servicios.

Instrucciones de uso inicial:

1. Autenticarse primero en usuarios-api para obtener un token JWT.

2. Usa el botón Authorize de Swagger UI y pega el token con formato Bearer.

3. Prueba los endpoints de ofertas, citas, fotos, catálogos y valoraciones según el rol requerido.

Servers

http://localhost:8082 - Generated server url

Authorize

1. Catalogos

Endpoints publicos para catalogos usados por ofertas de servicio

GET

/api/catalogos/categorias

Listar categorias de servicio

GET

/api/catalogos/tipos-precio

Listar tipos de precios

2. Estados

Endpoint de consulta para estados del flujo de citas

GET

/api/estados

Listar estados de cita

3. Ofertas de servicio

Endpoint para publicar, consultar, actualizar y eliminar ofertas de servicio

GET

/api/ofertas

Listar ofertas

POST

/api/ofertas

Crear oferta de servicio

GET

/api/ofertas/{id}

Buscar oferta por ID

PUT

/api/ofertas/{id}

Actualizar oferta por PUT

DELETE

/api/ofertas/{id}

Eliminar oferta

localhost:8083/swagger-ui/index.html/#

Swagger

API Docs

Explore

ConTrabajo - API de Comunicaciones

API de Comunicaciones

Documentación oficial de la API de comunicaciones.

Instrucciones de uso inicial:

1. Autenticarse primero en usuarios-api para obtener un token JWT.

2. Usa el botón Authorize de Swagger UI y pega el token con formato Bearer.

3. Prueba los endpoints de chats, mensajes y reportes según el rol requerido.

Servers

http://localhost:8083 - Generated server url

Authorize

1. Chats

Endpoint para iniciar chats, enviar mensajes y sincronizar estados de lectura

GET

/api/chats

Listar mis chats

PATCH

/api/chats/{idChat}/desactivar

Desactivar chat por ID

GET

/api/chats/{idChat}/historial

Obtener historial de chat

PATCH

/api/chats/{idChat}/leidos

Marcar mensajes como leídos

PATCH

/api/chats/{idChat}/recibidos

Marcar mensajes como recibidos

PATCH

/api/chats/{idChat}/vincular-cita

Vincular cita a chat

PATCH

/api/chats/desactivar

Desactivar chat por trabajador y oferta

POST

/api/chats/iniciar

Iniciar o recuperar chat

POST

/api/chats/mensaje

Enviar mensaje de chat

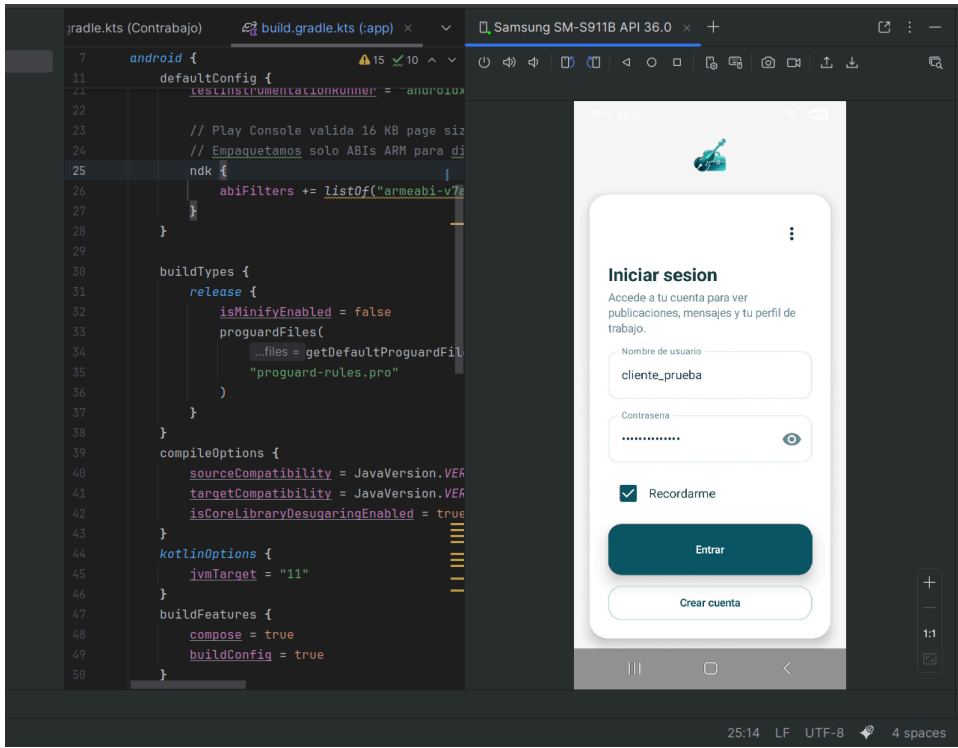
2. Reportes

Endpoints para crear reportes y gestionar revision de moderacion

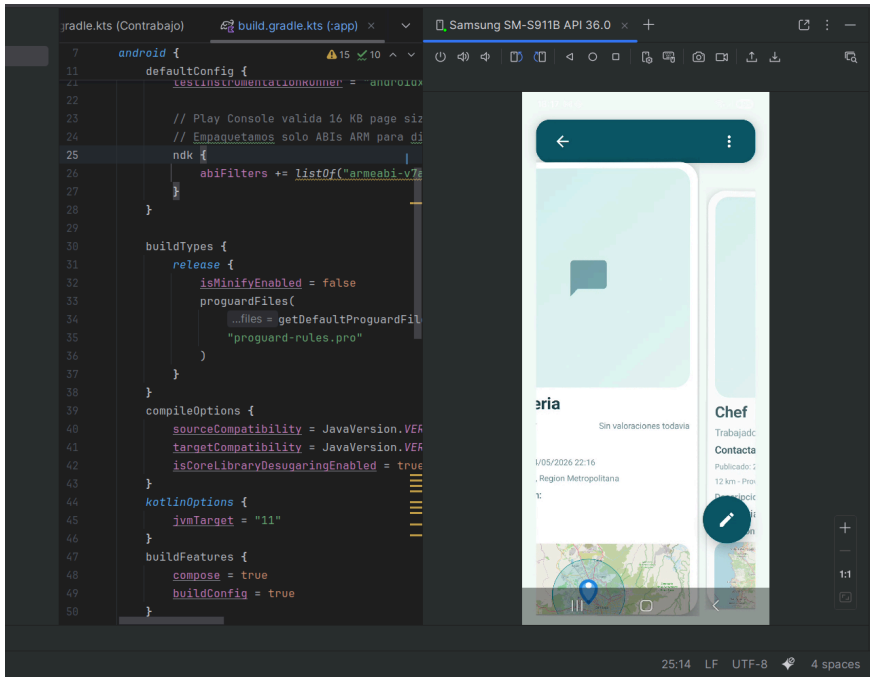
Matriz de Flujos de Interfaz Implementados (Capa Frontend)

Por su parte, la interfaz nativa implementada en Kotlin sobre Android Studio cuenta con las principales vistas de interacción del usuario totalmente funcionales e integradas con las reglas de negocio del Backend mediante peticiones REST asíncronas controladas por corrutinas:

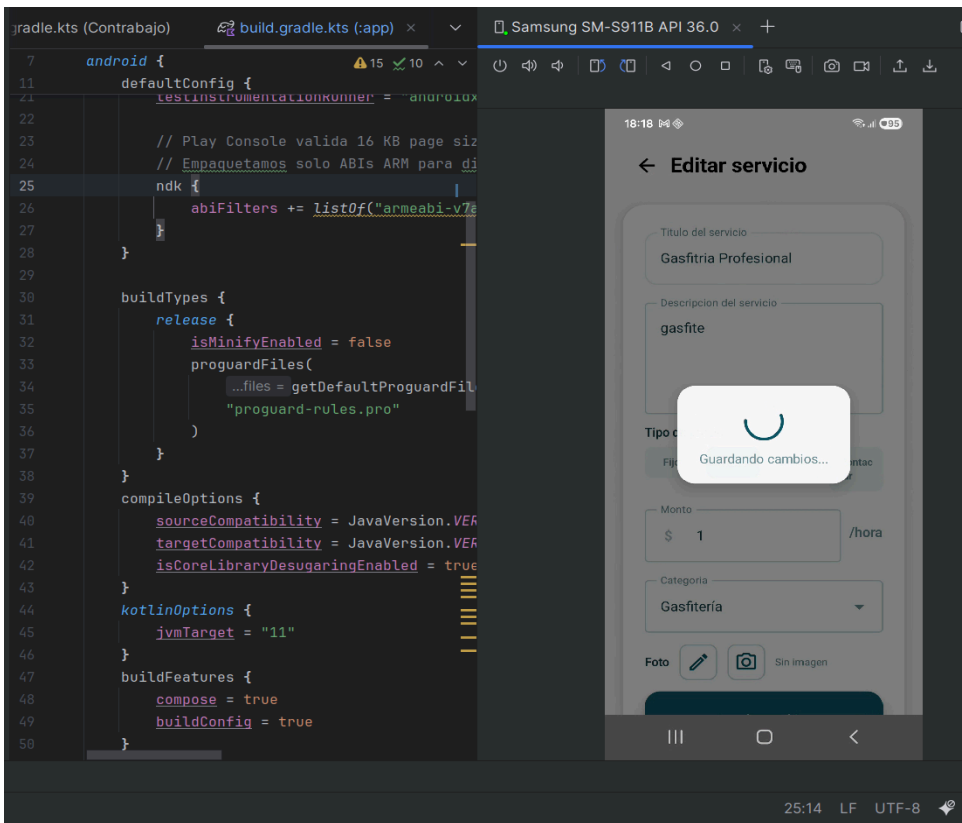
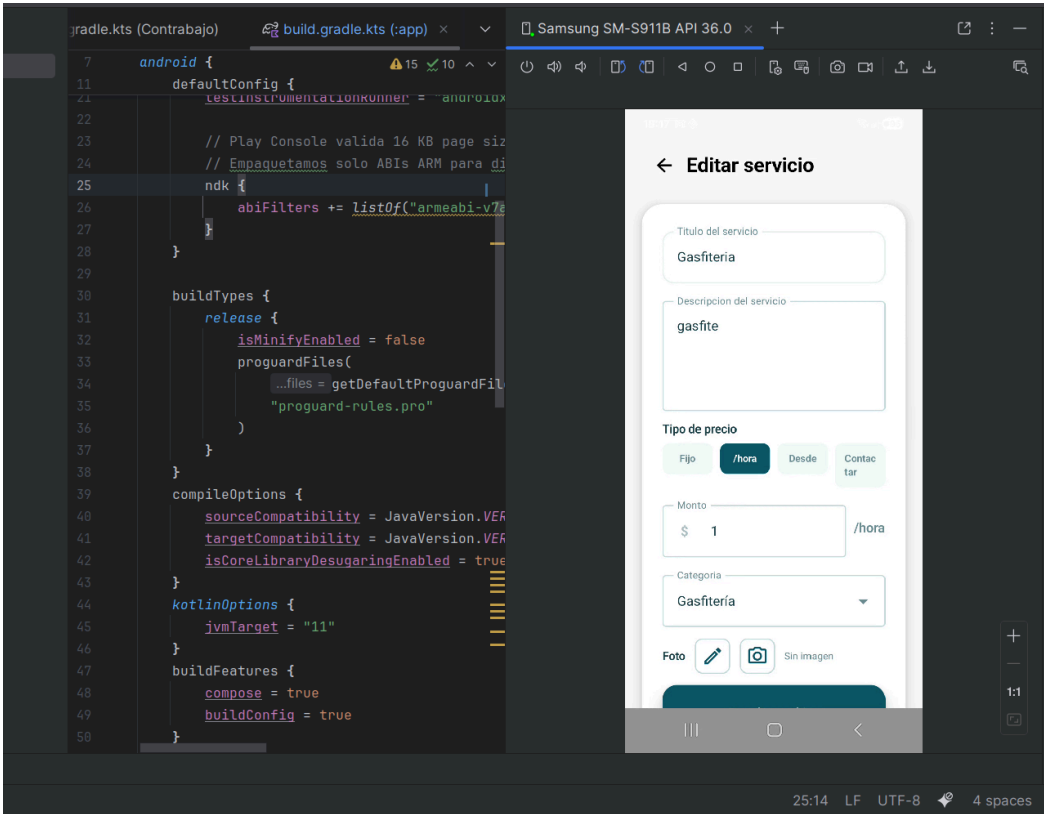
- **Pantalla de Login y Registro Nativo:** Formularios reactivos con captura estructurada de datos demográficos y validación en tiempo real de campos vacíos. Al validar las credenciales con éxito en el backend, almacena el Token JWT de forma local y segura en el almacenamiento del dispositivo.



- **Dashboard / Buscador Principal de Ofertas:** Despliegue dinámico de listas mediante componentes visuales (Cards), mapeando en tiempo real el catálogo de servicios de técnicos disponibles con adaptaciones visuales reactivas según el rol de cliente o trabajador.



- **Pantalla Editor de Servicio:** Formulario exclusivo del rol trabajador para ingresar títulos, descripciones y precios de nuevas ofertas laborales y enviarlas mediante solicitudes HTTP POST hacia la capa del servidor.



6. METODOLOGÍA Y PLANIFICACIÓN (IL 1.4)

La complejidad de desarrollar una solución nativa con arquitectura de microservicios requiere un marco de trabajo que permita la adaptabilidad y la entrega continua. Se ha seleccionado **Scrum** como marco de trabajo ágil, asegurando que el proyecto avance de forma incremental hacia los objetivos semestrales.

6.1 Marco de Trabajo Ágil

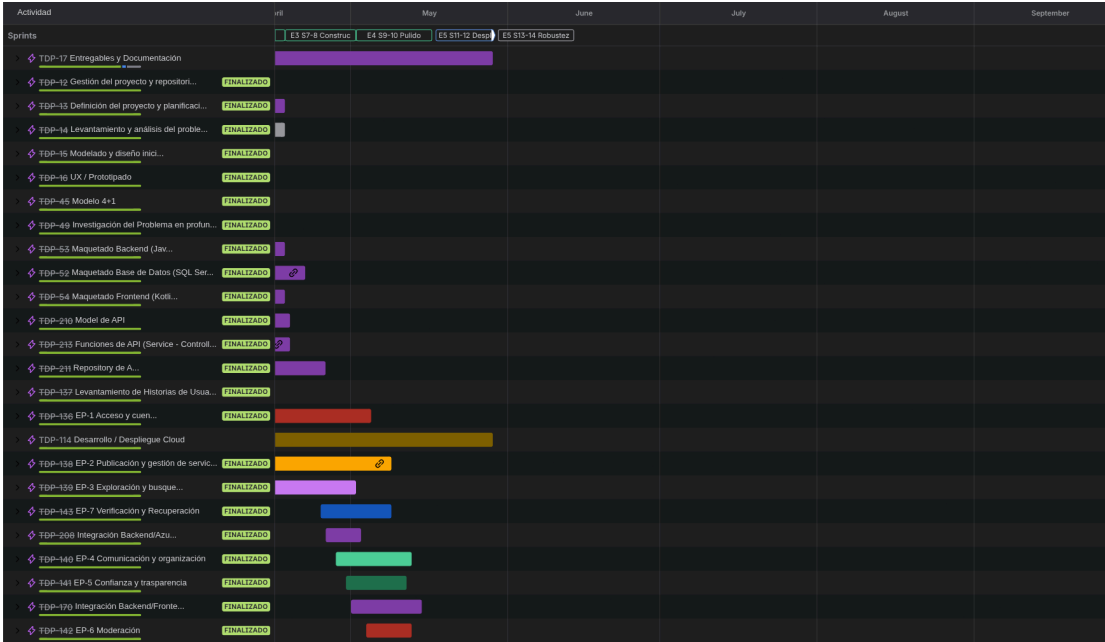
El equipo adoptará los siguientes ritos de Scrum para garantizar la transparencia y la inspección del avance:

- **Sprints (Ciclos de 2 semanas):** El desarrollo se dividirá en bloques de tiempo fijos. Cada Sprint entregará un incremento funcional (ej. Sprint 1: Módulo de Login y Conexión a Base de Datos).
- **Daily Scrum:** Reuniones diarias de 15 minutos para sincronizar tareas entre el desarrollo de Frontend (Kotlin) y Backend (Java), identificando bloqueos técnicos de forma temprana.
- **Sprint Planning:** Definición de las historias de usuario del *Backlog* que se comprometerán en cada iteración, priorizando la seguridad, el flujo de contacto y la coordinación de citas de servicio.
- **Sprint Review & Retrospective:** Al finalizar cada ciclo, se presentará el avance al docente y el equipo analizará oportunidades de mejora en el proceso de desarrollo y despliegue en Azure.

6.2 Mapeo de Iteraciones y Cronograma de Trabajo

Para garantizar el cumplimiento estricto de los plazos institucionales y coordinar el desarrollo en paralelo de los microservicios, las definiciones conceptuales de **Scrum** descritas anteriormente han sido traducidas a una línea de tiempo estructurada. Dado que los ritos ágiles mitigan riesgos de retraso mediante revisiones constantes, el equipo ha sincronizado las entregas parciales con las fases del calendario académico.

De este modo, las épicas complejas del **Backlog** (tales como la orquestación del entorno y la persistencia independiente de datos) se desglosan en actividades secuenciales y dependencias críticas. Este ordenamiento temporal, que abarca el diseño arquitectónico, el desarrollo iterativo por componentes, la fase intensiva de pruebas funcionales y la estabilización del software, se consolida formalmente en la herramienta de planificación que se presenta a continuación.



La gráfica evidencia una planificación ágil estructurada cronológicamente a través de bloques incrementales de desarrollo (Sprints), cubriendo desde las fases de fundamentación teórica hasta la consolidación técnica. El estado del proyecto se certifica de la siguiente manera:

- **Fase Inicial y Diseño (Hitos Base):** Se observan completadas con éxito todas las tareas de gobernanza y arquitectura inicial, tales como el *Modelado 4+1* (*TDP-45*), la investigación profunda del problema y el maquetado inicial tanto del Backend en Java como de la Base de Datos en SQL Server.
- **Fase de Construcción e Integración Coherente:** El equipo consolidó de manera progresiva las épicas nucleares del sistema. Destacan como finalizados los componentes de *Acceso y Cuentas* (*TDP-136*), la *Publicación y Gestión de Servicios* (*TDP-138*), y los módulos asíncronos críticos de *Comunicación y Organización* (*TDP-140*) que dan soporte al chat.
- **Hitos de Despliegue e Infraestructura Cloud:** Se valida el cierre definitivo de las tareas de *Desarrollo / Despliegue Cloud* (*TDP-114*) e *Integración Backend/Azure* (*TDP-208*). Esto demuestra en la línea de tiempo el traspaso exitoso de los microservicios desde el ambiente Docker local hacia la infraestructura IaaS productiva (Máquina Virtual en Azure).
- **Robustez y Verificación:** Se constata el cierre de la épica de *Verificación y Recuperación* (*TDP-143*), la cual respalda directamente los mecanismos de *Backup & Restore* mediante scripts automatizados que detallamos en las secciones técnicas anteriores.

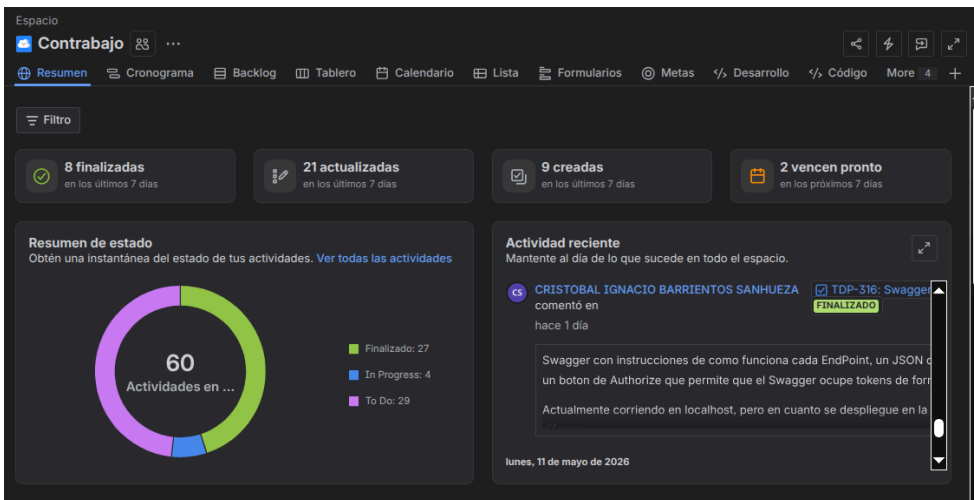
El cumplimiento riguroso del cronograma expuesto en la **Carta Gantt** no solo depende de la asignación temporal de tareas, sino también de una estrategia de control que mitigue de forma temprana los riesgos de integración e inconsistencia de datos propios de un ecosistema modular.

7. MECANISMOS DE CONTROL Y CALIDAD

7.1 Certificación de Avances

La complejidad técnica de integrar una aplicación móvil nativa en Kotlin con una arquitectura de microservicios distribuida y orquestada en contenedores requiere mecanismos de control de calidad rigurosos. Para asegurar que el avance del desarrollo se alinee con las expectativas académicas y los requisitos de negocio, el equipo implementará los siguientes protocolos de validación y certificación de entregables:

- **Demostraciones de Ciclo Corto (Sprint Demos):** Al finalizar cada hito crítico del proyecto (como la integración de las APIs con la persistencia en SQL Server o el despliegue en la máquina virtual IaaS de Azure), se realizará una presentación funcional directamente a la docente guía. Este rito ágil permite obtener retroalimentación temprana y corregir desviaciones en la lógica de negocio antes de proceder al siguiente ciclo de desarrollo.
- **Matriz de Aceptación de Requerimientos:** Se llevará un control estricto de avance mediante listas de verificación (*Checklists*) vinculadas de forma directa a los Requisitos Funcionales (RF) definidos en la fase de análisis. Un requerimiento se considerará formalmente "Certificado" únicamente cuando supere las pruebas operativas y de validación correspondientes en el entorno local.
- **Validación de Prototipado y Flujos de Usuario (UI/UX):** Antes de proceder con la codificación definitiva de las interfaces en Android Studio, se valorarán las maquetas y flujos de usuario. El objetivo de este control es certificar que la navegación y el comportamiento reactivo de la aplicación móvil sean intuitivos para el cliente final antes de acoplarlos con los microservicios del backend.



Con el propósito de respaldar de manera empírica la correcta aplicación de estos protocolos de calidad, se trabajó en **Jira** con el fin de mantener un seguimiento de las tareas y cada progreso correspondiente.

7.2 Plan de Pruebas Inicial

Para certificar la calidad, estabilidad y seguridad de la plataforma Contrabajo, el equipo definió un plan de pruebas acotado y sistemático enfocado en tres dimensiones operacionales: la ejecución correcta del flujo principal (*happy path*), la validación estricta de entradas de datos y la verificación de las reglas de autorización del negocio.

- **Alcance del Entorno de Pruebas:**
 - **Capa Backend:** Verificación de las especificaciones de contratos de software de los tres microservicios (usuarios-api, servicios-api, comunicaciones-api) de forma local y en el entorno IaaS.
 - **Capa Frontend Móvil:** Aseguramiento de la integración cliente-servidor mediante *smoke-tests* manuales sobre un emulador Android (AVD) conectado a la red del orquestador.
- **Clasificación Metodológica de Pruebas (Matriz de Tipos):**

Tipo de Prueba	Enfoque de Control en el Proyecto	Ejemplo Práctico de Aplicación
Operativa	Comprueba que el sistema cumple su función esperada ante flujos correctos de datos.	Login de usuario devuelve un código HTTP 200 OK y el Token JWT correspondiente.
Validación	Verifica que el backend rechaza datos de entrada incorrectos o corruptos.	El registro de una cuenta con un correo ya existente es denegado con un código 4xx.
Verificación	Valida restricciones de negocio, privilegios de acceso y seguridad criptográfica.	Un usuario con rol CLIENTE tiene prohibido (403 Forbidden) publicar una oferta técnica.

- **Estrategia Tecnológica y Herramientas:**
 - **Swagger UI / Postman:** Interfaces utilizadas para el testeo directo de las APIs distribuidas en sus respectivos puertos de escucha (:8081, :8082, :8083).
 - **Docker Compose:** Utilizado para inicializar y sincronizar en un solo comando (`docker compose up --build`) todo el stack del servidor, aislando la persistencia transaccional.

7.3 Plan de Pruebas Detallado

7.3.1 Pruebas Operativas, de Validación y Verificación

A continuación, se expone la matriz de los casos de prueba más críticos extraídos del plan maestro del proyecto. Estos casos representan los puntos de quiebre lógicos más importantes del ecosistema microservicios-móvil:

Matriz de Casos de Prueba Esenciales

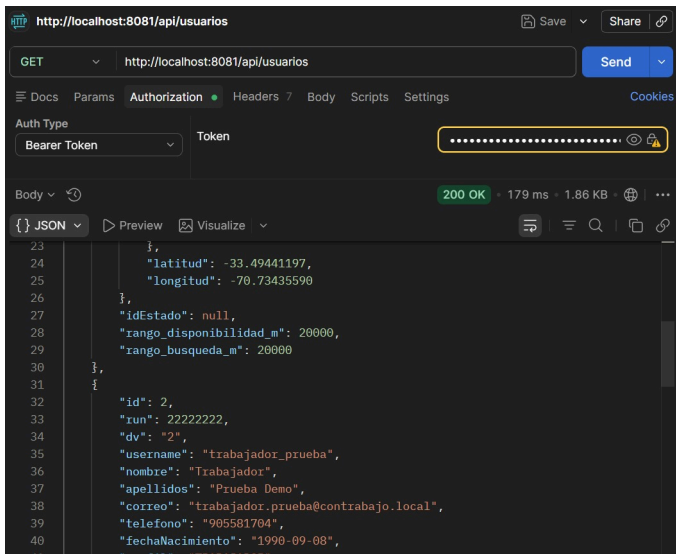
ID	Componente / API	Tipo	Método / Endpoint	Estímulo (Entrada)	Resultado Esperado
U-01	usuarios-api (:8081)	Operativa	POST /api/auth/login	Credenciales correctas de un técnico.	Código 200 OK + Emisión de Token JWT.
U-04	usuarios-api (:8081)	Validación	POST /api/usuarios/regar	RUN o correo electrónico duplicado.	Código 4xx + Mensaje claro de duplicidad.
S-01	servicios-api (:8082)	Operativa	POST /api/ofertas	Token TRABAJADOR + JSON de servicio.	Código 201 Created; registro en BD.
S-02	servicios-api (:8082)	Verificación	POST /api/ofertas	Token con privilegios de CLIENTE.	Código 403 Forbidden (Bloqueo de seguridad).
C-03	comunicaciones-api (:8083)	Operativa	POST /api/reportes	Token CLIENTE + ID de oferta ofensiva.	Código 201 Created (Persistencia de denuncia).
E2E-1	Integración Completa	E2E	Flujo transversal distribuido	Registro, publicación, asignación y fin de cita.	Tránsito transaccional exitoso a FINALIZADO.
AND-3	Contrabajo-App	Front (Manual)	PantallaEditorServicio	Publicación desde formulario nativo Kotlin.	

7.3.2 Registro de Evidencias Empíricas de Ejecución

En cumplimiento estricto de las directrices del indicador de la pauta de evaluación, se documenta la ejecución exitosa de las pruebas mediante tres evidencias físicas clave extraídas directamente del ambiente de desarrollo:

Evidencia de Pruebas de Validación (Contratos de API en Postman/Swagger)

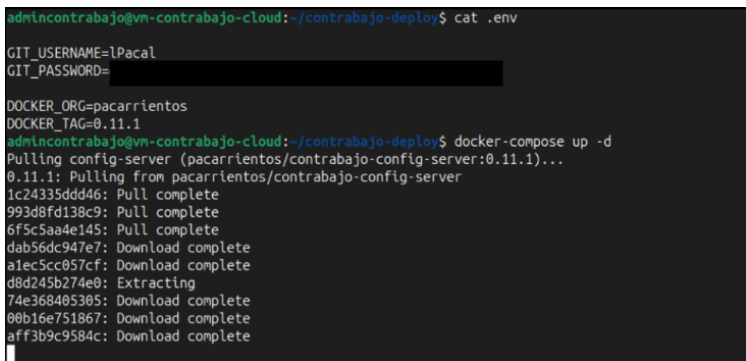
Esta captura demuestra de manera empírica que los endpoints validan correctamente las solicitudes de red, controlan las excepciones y estructuran las respuestas JSON de seguridad de acuerdo al diseño técnico.



Evidencia de prueba de validación ejecutada en entorno controlado, certificando la estructura de respuesta del servidor y la entrega de firmas criptográficas de sesión.

Evidencia de Pruebas de Verificación (Aprovisionamiento de la Infraestructura)

Esta evidencia demuestra que el ambiente de pruebas simula de forma exacta el entorno productivo Linux a través de la correcta orquestación de servicios y el aislamiento de las variables de red del servidor.



Verificación del motor virtualizado, comprobando la estabilidad de la red interna aislada y el levantamiento del entorno cloud.

Evidencia de Pruebas Operativas (Integración del Cliente Android)

Esta captura comprueba el avance real y operativo del producto de software. Certifica que la aplicación móvil nativa en Kotlin interactúa con el backend asincrónicamente a través de peticiones HTTP/WebSockets sin generar fallos ni congelamientos de interfaz.

```
● tomy@atlas:~/Downloads/contrabajo/ms/config/contrabajo-deploy$ docker logs contrabajo-comunicaciones-api | grep -iE "websocket|stomp|broker"
2026-05-24T21:12:31.693Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : Starting...
2026-05-24T21:12:31.695Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : BrokerAvailabilityEvent[available=true, SimpleBrokerMessageHandler [org.springframework.messaging.simp.broker.DefaultSubscriptionRegistry@36f291c7]]
2026-05-24T21:12:31.697Z INFO 1 --- [comunicaciones_api] [main] o.s.m.s.b.SimpleBrokerMessageHandler : Started.
2026-05-24T21:13:31.660Z INFO 1 --- [comunicaciones_api] [MessageBroker-1] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[0 current WS(0)-HttpStream(0)-HttpPoll(0), 0 total, 0 closed abnormally (0 connect failure, 0 send limit, 0 transport error)], stompSubProtocol[processed CONNECT(0)-CONNECTED(0)-DISCONNECT(0)], stompBrokerRelay[null], inboundChannel[pool size = 0, active threads = 0, queued tasks = 0, completed tasks = 0], outboundChannel[pool size = 0, active threads = 0, queued tasks = 0, completed tasks = 0], sockJsScheduler[pool size = 1, active threads = 1, queued tasks = 0, completed tasks = 0]
```

Evidencia de prueba operativa de extremo a extremo en el frontend móvil, validando el consumo correcto de servicios y la renderización reactiva de las reglas de negocio.

7.3.3 Evidencia de Pruebas Unitarias Automatizadas y Cobertura de Código

Como complemento a las pruebas operativas e integradas de la plataforma, el equipo *Zerberus Softwork* implementó una batería de pruebas unitarias automatizadas. Estas pruebas tienen como finalidad aislar la lógica de negocio nuclear alojada en los servicios (**@Service**) de los microservicios, asegurando que ante mutaciones en el código fuente, el comportamiento algorítmico permanezca invariante y libre de regresiones.

Implementación Técnica del Aislamiento (Estrategia de Mocks)

Para garantizar el principio de aislamiento estricto (omitiendo conexiones a la base de datos SQL Server o llamadas de red del Config Server), se utilizó la biblioteca **Mockito**. Mediante las anotaciones **@Mock** y **@InjectMocks**, se simula de manera determinista el comportamiento de la capa de persistencia (Spring Data JPA), permitiendo evaluar exclusivamente las reglas de negocio críticas como la encriptación de contraseñas y la firma de tokens JWT.

A continuación, se expone un extracto del código fuente diseñado para la prueba unitaria del caso feliz de autenticación en usuarios-api:

```
// =====
// Test: Login exitoso
// =====
@Test
void testLogin_Exitoso() {
    // Arrange
    when(usuarioRepository.findByUsername("usuario_test")).thenReturn(Optional.of(usuarioMock));
    when(passwordEncoder.matches("password123", "hashedPassword")).thenReturn(true);
    when(sessionRepository.findByIdAndActivaTrue(1)).thenReturn(List.of());
    when(jwtUtil.generarToken(usuarioMock)).thenReturn(jwtToken);
    when(sessionRepository.save(any(SesionUsuario.class))).thenAnswer(invocation -> invocation.getArgument(0));

    // Act
    LoginResponseDTO resultado = authService.login(loginDTO);

    // Assert
    assertNotNull(resultado);
    assertEquals("usuario_test", resultado.getUsuario().getUsername());
    assertEquals(jwtToken, resultado.getToken());
    verify(usuarioRepository, times(1)).findByUsername("usuario_test");
    verify(passwordEncoder, times(1)).matches("password123", "hashedPassword");
    verify(jwtUtil, times(1)).generarToken(usuarioMock);
    verify(sessionRepository, times(1)).save(any(SesionUsuario.class));
}
```

Registro de Ejecución y Barras de Éxito (Green Bar)

La ejecución coordinada de las pruebas automatizadas se realiza de forma directa en el entorno de desarrollo a través de las herramientas nativas del IDE (IntelliJ IDEA para el Backend y Android Studio para la aplicación móvil Kotlin).

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 91, Failures: 0, Errors: 0, Skipped: 1
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 17.895 s
[INFO] Finished at: 2026-05-25T02:18:34-04:00
[INFO] -----
C:\Users\barri\OneDrive\Escritorio\APIs Act\contrabajo-servicios-api>
```

```
[INFO]
[INFO] Results:
[INFO]
[WARNING] Tests run: 78, Failures: 0, Errors: 0, Skipped: 1
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 24.264 s
[INFO] Finished at: 2026-05-25T02:20:55-04:00
[INFO] -----
C:\Users\barri\OneDrive\Escritorio\APIs Act\contrabajo-usuarios-api>
```

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 39, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 14.262 s
[INFO] Finished at: 2026-05-25T02:27:02-04:00
[INFO] -----
C:\Users\barri\OneDrive\Escritorio\APIs Act\contrabajo-comunicaciones-api>
```

La incorporación exitosa de la sección 7.3 concluye que el software cumple con los tres niveles de la pirámide de pruebas. El acoplamiento armonioso entre las pruebas analizadas demuestra que:

- 1. Las funciones individuales son sólidas de manera aislada (**Pruebas Unitarias**).
- 2. Los contratos e intercambio de información estructurada JSON responden según el diseño de arquitectura (**Pruebas de Validación**).
- 3. La experiencia integrada del cliente final en el dispositivo móvil se ejecuta con fluidez sobre la infraestructura aprovisionada (**Pruebas Operativas**).

8. CONCLUSIÓN

El proyecto Contrabajo se consolida como una respuesta tecnológica de alto nivel frente a la informalidad y los riesgos críticos de seguridad que hoy predominan en el mercado de servicios técnicos domésticos y urbanos. Nuestra propuesta ofrece una solución integral que centraliza y dinamiza la interacción entre prestadores y clientes, transformando procesos análogos y fragmentados en un ecosistema digital eficiente, seguro y auditable.

Diseñamos esta plataforma bajo estrictos estándares de arquitectura de software, priorizando el desacoplamiento de componentes, la integridad distribuida de los datos y la validación de identidad mediante un modelo de autenticación *stateless* basado en tokens JWT. Construida sobre un stack tecnológico robusto compuesto por un backend de microservicios en Java (Spring Boot), un cliente frontend nativo en Kotlin y una estrategia de despliegue contenedorizada bajo **Infraestructura como Servicio (IaaS) en una Máquina Virtual de Microsoft Azure**, la solución permite una búsqueda ágil de servicios, navegación reactiva, **mensajería instantánea bidireccional en tiempo real** y una coordinación eficiente de citas. Todo esto con un objetivo fundamental: mitigar la pérdida de tiempo, maximizar la confianza mutua y profesionalizar de manera definitiva la conexión entre los oficios técnicos y la comunidad que los requiere.

9. BIBLIOGRAFÍA

Informe de investigación para sustentar “Contrabajo”

https://docs.google.com/document/d/1tXr-aRO0_-hAVTh-TIMFJRMxr9sTt-j/edit?usp=sharing&oid=105829578132632956326&rtpof=true&sd=true

Instituto Nacional de Estadísticas. (2024). *Empleo trimestral N° 28: Informalidad y condiciones laborales (julio-septiembre 2024)*.

<https://www.ine.gob.cl/docs/default-source/informalidad-y-condiciones-laborales/boletines/2024/ene-informalidad-28.pdf>

Ministerio de Economía, Fomento y Turismo, & Instituto Nacional de Estadísticas. (2023).

Síntesis de resultados EME-VII.

<https://www.economia.gob.cl/wp-content/uploads/2025/11/sintesis-de-resultados-eme-7.pdf>

Comisión Nacional de Evaluación y Productividad. (2023). *Informe preliminar brecha de uso: Productividad en telecomunicaciones*.

<https://cnepe.cl/wp-content/uploads/2023/10/Informe-preliminar-brecha-de-uso.pdf>

Comisión Económica para América Latina y el Caribe, & Organización Internacional del Trabajo. (2023). *Coyuntura laboral en América Latina y el Caribe N° 28: Hacia la creación de mejor empleo en la pospandemia*.

https://repositorio.cepal.org/bitstream/handle/11362/48987/1/S2300435_es.pdf

ChileAtiende. (2026). *Buscador de instaladores eléctricos y de gas autorizados por la SEC*.

<https://www.chileatiende.gob.cl/fichas/67276/1/pdf>

Servicio Nacional del Consumidor. (s. f.). *Servicios técnicos: Información de mercados y productos*. <https://www.sernac.cl/portal/619/w3-article-1373.html>

Organización Internacional del Trabajo. (2023). *Generative AI and jobs: A global analysis of potential effects on job quantity and quality* (Working Paper No. 96).

https://www.ilo.org/sites/default/files/2024-07/WP96_web.pdf

Organisation for Economic Co-operation and Development. (2023). *OECD employment outlook 2023: Artificial intelligence and the labour market*.

https://www.oecd.org/content/dam/oecd/en/publications/reports/2023/07/oecd-employment-outlook-2023_904bcef3/08785bba-en.pdf

10. ANEXOS

Figura. Diagrama de casos de uso – Gestión de Usuarios:

https://drive.google.com/file/d/1sdiSw50VwNelqXjUPfTa9ZsOaKPEGeg0/view?usp=drive_link

Figura. Diagrama de casos de uso – Servicios

https://drive.google.com/file/d/1fzfAlpbIUkCsb1eaYfUkl8XvgGrB3Mxr/view?usp=drive_link

Figura. Diagrama de casos de uso – Comunicación

https://drive.google.com/file/d/18_wEYJraJB-NKiuzPvGSLMOFfo-RkWx/view?usp=drive_link

Figura. Diagrama de casos de uso – Administración

https://drive.google.com/file/d/1T7PzPR8CtC8pgbPM_SUEQm30oXkjV_6w/view?usp=drive_link

Entrevistas y Mapas de empatía (levantamiento de información)

https://docs.google.com/document/d/1Q4wIbf73X5pPKUOCUgxtKIdR5fgw9eC-5fsuzz_61lQ/edit?usp=sharing

Mapas de empatía:

https://docs.google.com/document/d/1ljGilfXJ4TkUP4qJm46q6_zsXG7WCQFAKziPoi81gsw/edit?usp=sharing

Historias de usuario:

<https://docs.google.com/spreadsheets/d/1HVRJ2BH6zM1vXKWc2oYqZTxSYvqWZHYi8eXaAzG5kq8/edit?usp=sharing>